第一章　马尔可夫决策过程 MDP

马尔可夫决策过程是描述随机动态系统的一般框架，它不局限于强化学习，而是强化学习需要依赖这个框架。马尔可夫决策涉及以下关键因素：

1. 集合：
   - 状态空间：所有状态的集合，记为 $\mathcal{S}$.
   - 动作空间：与每个状态 $s \in \mathcal{S}$ 相关联的所有动作集合，记为 $\mathcal{A}(s)$.
   - 奖励集合：与 (s, a) 相关联的所有奖励的集合，记为 $\mathcal{R}(s, a)$.
2. 模型：
   - 状态转移概率：在状态 s 采取动作 a 时，智能体转移到状态 $s^{'}$ 的概率为 $p(s^{'} \mid s, a)$。对于任意 (s, a)，都有
   $$\sum_{s^{'} \in \mathcal{S}} p(s^{'} \mid s, a) = 1.$$
   - 奖励概率：在状态 s 采取动作 a 时，智能体获得奖励 r 的概率是 $p(r \mid s, a)$。对于任意 (s, a), 都有
   $$\sum_{r \in \mathcal{R}(s,a)} p(r \mid s, a) = 1$$
3. 策略：在状态 s, 智能体采取动作 a 的概率是 $\pi(a \mid s)$。对于任意 $s \in \mathcal{S}$, 都有
   $$\sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1.$$

其中最重要的是马尔科夫性是指随机过程中的无记忆性质，在数学上可以表示为

$$p(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \cdots, s_0, a_0) = p(s_{t+t} \mid s_t, a_t)$$

$$p(r_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \cdots, s_0, a_0) = p(s_{t+t} \mid r_t, a_t)$$

在马尔可夫决策过程中大多是状态转移函数和奖励函数具有马尔可夫性。

第二章　贝尔曼方程

我们可以通过计算折扣汇报来评价不同策略的好坏。其中折扣回报的计算方式有两种：

1. 根据定义：回报等于沿轨迹收集的所有奖励的折扣总和。
2. 自举法 (Bootstrapping)：将方程化简为矩阵向量形式的线性方程组:

$$v = r + \gamma P v$$

其中 $v, r \in \mathbb{R}^n$, $P \in \mathbb{R}^{n\times n}$。这个线性方程组是由当前状态的折扣回报与一步转移之后的折扣回报建立起来的。求解每个初始状态的折扣回报 v_t 等价于求解上述线性方程组。

2.1　状态值

前面虽然提到回报可以用来评价策略，但是回报并不适用一般化的随机情况，因为从一个状态出发可能会得到不同的轨迹和回报，为此我们引入状态值的概念。

我们首先引入一些符号。在时刻 $t = 0, 1, 2, \cdots$，智能体处于状态 S_t，按照策略 π 采取动作 A_t，转移到下一个状态是 S_{t+1}，获得的即时奖励是 R_{t+1}。这个过程通常被简洁的表达为

$$S_t \xrightarrow{A_t} S_{t+1}, R_{t+1}.$$

这里 $S_t, S_{t+1} \in \mathcal{S}$, $A_t \in \mathcal{A}(S_t)$, $R_{t+1} \in \mathcal{R}(S_t, A_t)$。请注意,这里的 S_t, A_t, S_{t+1}, R_{t+1} 都是随机变量。

从 t 时刻开始，我们可以得到一条包含一系列“状态-动作-奖励”的轨迹：

$$S_t \xrightarrow{A_t} S_{t+1}, R_{t+1} \xrightarrow{A_{t+1}} S_{t+2}, R_{t+2} \xrightarrow{A_t} S_{t+3}, R_{t+3}, \cdots$$

根据定义，沿这个轨迹得到的折扣回报是

$$G_t := R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+1+k}$$

其中 $\gamma \in (0,1)$ 是折扣因子，其目的是保证折扣回报收敛。请注意，G_t 也是一个随机变量, 因为它是 $R_{t+1}, R_{t+2}, \cdots$ 这些随机变量组合得到的。

由于 G_t 是一个随机变量，因此我们可以计算它的期望值为

$$v_\pi(s) := \mathbb{E}[G_t \mid S_t = s]$$

这里 $v_{\pi(s)}$ 称为状态值函数 (state-value function)。下面是对状态值的一些说明:

1. $v_\pi(s)$ 的值依赖于 s，即不同状态的状态值一般是不同的。
2. $v_\pi(s)$ 的值依赖于 π，即不同的策略对应的状态值一般是不同的。这是因为轨迹是根据策略 π 得到的，不同的策略可能导致不同的轨迹。
3. $v_\pi(s)$ 并不依赖于 t。不论 t 选取什么值得到的结果都是相同的，这本质是因为系统是平稳的。

“状态值” 与 “回报” 之间的关系：当策略和系统模型都是确定性时，从一个状态出发始终会得到相同的轨迹。此时，从一个状态出发得到的回报就等于状态值。相反，当策略和模型都是随机时，从一个状态出发可能产生不同的轨迹。此时，不同的轨迹的回报是不同的，而状态值就是这些回报的期望值。因此，状态值比回报更加一般化，能同时处理确定性和随机性的情况。虽然之前提到回报能用来评价策略的好坏，但是更一般化地应用状态值来评价策略。

2.2　贝尔曼方程

贝尔曼方程描述了所有状态值之间的关系。下面我们详细推导贝尔曼方程。

首先 G_t 可以重写为：

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

该方程建立了 G_t 与 G_{t+1} 之间的练习。因此，状态值可以写成

$$\begin{aligned} v_\pi(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}[G_{t+1} \mid S_t = s]. \end{aligned}$$

下面我们分别分析上述式子中的两项。

1. 第一项 $\mathbb{E}[R_{t+1} \mid S_t = s]$ 时即时奖励的期望值。基于全期望公式，这一项可以化简为：

$$\begin{aligned}\mathbb{E}[R_{t+1} \mid S_t = s] &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \mathbb{E}[R_{t+1} \mid S_t = s,\, A_t = a] \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{r \in \mathcal{R}} p(r \mid s, a) r.\end{aligned}$$

   这里 $\mathcal{A}$ 和 $\mathcal{R}$ 分别时可能的动作和可能的奖励的集合。对于不同的状态，$\mathcal{A}$ 可能是不同的；此时 $\mathcal{A}$ 可以表示为 $\mathcal{A}(s)$。类似地，$\mathcal{R}$ 也可能依赖于 (s, a)。

2. 第二项 $\mathbb{E}[G_{t+1} \mid S_t = s]$ 是未来奖励的期望。它可以计算为：

$$\begin{aligned}\mathbb{E}[G_{t+1} \mid S_t = s] &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1} \mid S_t = s,\, S_{t+1} = s^{'}] p(s^{'} \mid s) \\ &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1} \mid S_{t+1} = s^{'}] p(s^{'} \mid s) \quad \text{（马尔可夫性）} \\ &= \sum_{s' \in \mathcal{S}} v_\pi(s^{'}) p(s^{'} \mid s) \\ &= \sum_{s' \in \mathcal{S}} v_\pi(s^{'}) \sum_{a \in \mathcal{A}} p(s^{'} \mid s, a) \pi(a \mid s).\end{aligned}$$

   上述推导过程中的最后一步将 $p(s^{'} \mid s)$ 展开为 $\sum_{a \in \mathcal{A}} p(s^{'} \mid s, a) \pi(a \mid s)$ 是因为从 s 到 $s^{'}$ 的转移概率不能直接求出，它是由状态 s 根据策略采取动作 a 后，再根据模型得出状态 s 采取 a 动作后转移到 $s^{'}$ 的概率两者结合求出。

将两项的展开式带入原方程可得：

$$\begin{aligned}v_\pi(s) &= \mathbb{E}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}[G_{t+1} \mid S_t = s] \\ &= \underbrace{\sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{r \in \mathcal{R}} p(r \mid a, s) r}_{\text{即时奖励的期望}} + \underbrace{\gamma \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} p(s^{'} \mid s, a) v_\pi(s^{'})}_{\text{未来奖励的期望}} \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \left[\sum_{r \in \mathcal{R}} p(r \mid a, s) r + \gamma \sum_{s' \in \mathcal{S}} p(s^{'} \mid s, a) v_\pi(s^{'}) \right], \quad s \in \mathcal{S}\end{aligned}$$

上式就是大名鼎鼎的贝尔曼方程，它描述了不同状态值之间的关系，是设计和分析强化学习算法的基本工具。

贝尔曼方程乍一看很复杂，但实际上它的结构非常清晰。下面是对该方程的解释说明。

1. $v_\pi(s)$ 和 $v_\pi(s^{'})$ 是需要计算的状态值，是未知量。如何计算我们将在之后讲解。

2. $\pi(a \mid s)$ 是一个给定的策略，是已知量。贝尔曼方程一定是对应了一个特定的策略。求解贝尔曼方程从而得到状态值是一个策略评估的过程，这是许多强化学习算法中的核心步骤。

3. $p(r \mid s, a)$ 和 $p(s^{'} \mid s, a)$ 代表系统模型，这是可能是已知量也可能是未知量。我们先研究已知的情况，后面推广到不需要模型的情况。

2.3 矩阵向量形式

由于每一个状态都对应了式 (1) 中的方程，因此我们可以联立所有这些方程进而得到间接的**矩阵-向量形式**。为了推导出矩阵-向量形式，首先将贝尔曼方程重写为：

$$v_\pi(s) = r_\pi(s) + \gamma \sum_{s' \in \mathcal{S}} p_\pi(s^{'} \mid s) v_\pi(s^{'}), \tag{2}$$

其中

$$r_\pi(s) := \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{r \in \mathcal{R}} p(r \mid s, a) r,$$

$$p_\pi(s^{'} \mid s) := \sum_{a \in \mathcal{A}} \pi(a \mid s) p(s^{'} \mid s, a).$$

这里 $r_\pi(s)$ 表示即时奖励的期望，而 $p_\pi(s^{'} \mid s)$ 代表在策略 π 下从状态 s 经过一步转移到状态 $s^{'}$ 的概率。

为了写成矩阵向量形式，需要对状态进行编号。如果有 $n = |\mathcal{S}|$ 个状态，那么给这 n 个状态编号为 $\{s_1, s_2, \cdots, s_n\}$。对于状态 s_i，(2) 可以写成:

$$v_\pi(s_i) = r_\pi(s_i) + \gamma \sum_{s_i \in \mathcal{S}} p_\pi(s_j \mid s_i) v_\pi(s_j). \tag{3}$$

定义 $v_\pi = [v_\pi(s_1), \cdots, v_\pi(s_n)]^\top \in \mathbb{R}^n$，$r_\pi = [r_\pi(s_1), \cdots, r_\pi(s_n)] \in \mathbb{R}^n$，以及 $P_\pi \in \mathbb{R}^{n \times n}$，其中 P_π 满足 $[P_\pi]_{ij} = p_\pi(s_j \mid s_i)$。此时，(3) 可以用以下矩阵-向量形式来表示：

$$v_\pi = r_\pi + \gamma P_\pi v_\pi,$$

这里 v_π 是待解的未知量，而 γ, r_π, P_π 是已知量。

矩阵 P_π 有一些有趣的性质。第一，它是一个非负矩阵。第二，P_π 是一个随机矩阵，即每一行所有元素之和等一 1。

2.4 求解状态值

上一节我们用矩阵形式表示了贝尔曼方程，求出状态值函数的过程实际就是求解线性方程组的过程。对于求解线性方程组我们通常有两种思路–解析法和数值法。

2.4.1 解析法

由于 $v_\pi = r_\pi + \gamma P_\pi v_\pi$ 是一个简单的线性方程，我们不难得到其解析解:

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi,$$

下面给出矩阵 $(I - \gamma P_\pi)^{-1}$ 的几个性质:

1. $I - \gamma P_\pi$ 是可逆的。

   因为 P_π 是随机矩阵，所以对于任意的第 k 行元素，满足:

   $$\sum_{i=1}^{n} p_\pi(s_i \mid s_k) = 1$$

   因为 $\gamma < 1$，所以 $\forall k \in \{1, 2, \cdots, n\}$:

   $$\sum_{i=1}^{n} \gamma p_\pi(s_i \mid s_k) < 1 \Longrightarrow \sum_{i \neq k}^{n} \gamma p_\pi(s_i \mid s_k) < 1 - \gamma p_\pi(s_k \mid s_k), \tag{4}$$

   根据 Gershgorin 圆盘定理我们可知，所有的特征值落在以以 $[I - \gamma P_\pi]_{ii} = 1 - \gamma p_\pi(s_i \mid s_i)$ 为中心，$\sum_{j \neq i} [I - \gamma P_\pi]_{ij} = -\sum_{j \neq i} \gamma p_\pi(s_j \mid s_i)$ 为半径的 Gershgonrin 圆盘中。由 (4) 可得，圆盘中心到原点的距离大于半径，所以一定不存在零特征值，也就说明矩阵 $I - \gamma P_\pi$ 一定可逆。

2. $(I - \gamma P_\pi)^{-1} \geq I$，即矩阵 $(I - \gamma P_\pi)^{-1}$ 的每一个元素都是大于或等于零的。这是因为 P_π 的元素都是非负的，考虑矩阵逆的级数展开可得:

   $$(I - \gamma P_\pi)^{-1} = I + \gamma P_\pi + \gamma^2 P_\pi^2 + \cdots \geq I \geq 0$$

3. 对于任何向量 $r \geq 0$，有 $(I - \gamma P_\pi)^{-1} r \geq r \geq 0$。这个性质可以由第二个性质得出:

   $$(I - \gamma P_\pi)^{-1} \geq I \Rightarrow (I - \gamma P_\pi)^{-1} - I \geq 0 \Rightarrow [(I - \gamma P_\pi)^{-1} - I] r \geq 0$$

在实际工程应用中使用 **LU 分解**来直接求解线性方程组。

2.4.2 数值解

虽然解析解对于理论分析非常有用，但因为它涉及矩阵的逆运算，因此任需要复杂的数值算法来计算。实际上我们可以用迭代法对贝尔曼方程进行数值计算：

$$v_{k+1} = r_\pi + \gamma P_\pi v_k, \quad k = 0, 1, 2, \cdots \tag{5}$$

从一个初始猜测值 $v_0 \in \mathbb{R}^n$ 出发，该算法会给出一个序列 $\{v_0, v_1, v_2, \cdots\}$ 并且该序列会收敛到真实的状态值，即

$$v_k \to v_\pi = (I - \gamma P_\pi)^{-1} r_\pi, \quad \text{随着} k \to \infty.$$

证明. 假设最后的真实状态值为 v_π，定义误差 $\delta_k = v_k - v_\pi$，将 $\delta_{k+1} = v_{k+1} - v_\pi$ 和 $\delta_k = v_k - v_\pi$ 带入 (5) 式：

$$v_\pi + \delta_{k+1} = r_\pi + \gamma P_\pi (v_\pi + \delta_k)$$

可以变化为

$$\begin{aligned}\delta_{k+1} &= (r_\pi - v_\pi + \gamma P_\pi v_\pi) + \gamma P_\pi \delta_k \\ &= \gamma P_k \delta_k\end{aligned}$$

由上式进一步可得:

$$\delta_{k+1} = \gamma P_\pi \delta_k = \gamma^2 P_\pi^2 \delta_{k-1} = \cdots = \gamma^{k+1} P_\pi^{k+1} \delta_0$$

因为 $\gamma < 1$ 并且 P_π 中的每一个元素都小于等于 1，所以 $\delta_{k+1} \to 0$。 □

2.4.3 动作值

针对一个状态-动作配对 (s, a)，其动作值定义为

$$q_\pi(s, a) := \mathbb{E}[G_t \mid S_t = s,\ A_t = a].$$

上式表明动作值被定义为在状态采取一个动作之后获得的回报的期望值。

动作值与状态值有什么关系呢?

1. 首先，根据条件期望的性质可得

$$\underbrace{\mathbb{E}[G_t \mid S_t = a]}_{v_{\pi(s)}} = \sum_{a \in \mathcal{A}} \underbrace{\mathbb{E}[G_t] \mid S_t = s, A_t = a]}_{q_\pi(s,a)} \pi(a \mid s)$$

上式可以简化为

$$\begin{aligned}v_\pi(s) &= \sum_{a \in \mathcal{A}} \pi(a \mid s) q_\pi(s, a) \\ &= \mathbb{E}_{A_t \sim \pi(s)}[q_\pi(s, A_t)].\end{aligned}$$

由上式可以看出，状态值是该状态对应的动作值得期望值。

2. 第二，因为状态值有如下表示：

$$v_\pi(s) = \sum_{a\in\mathcal{A}} \pi(a \mid s) \left[\sum_{r\in\mathcal{R}} p(r \mid s,a) r + \gamma \sum_{s'\in\mathcal{S}} p(s' \mid s,a) v_\pi(s') \right].$$

将其与上式对比可得:

$$\begin{aligned} q_\pi(s,a) &= \sum_{r\in\mathcal{R}} p(r \mid s,a) r + \gamma \sum_{s'\in\mathcal{S}} p(s' \mid s,a) v_\pi(s') \\ &= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a]. \end{aligned}$$

上述式子中的期望只是为了形式上简便，实际计算中还是用展开式。

第三章　最优状态值与贝尔曼最优方程

3.1　最优状态值和最优策略

定义　3.1 (最优策略 optimal policy)**.** 考虑两个给定的策略 π_1 和 π_2。如果对于任意状态值 $s \in \mathcal{S}$，π_1 的状态值都大于或等于 π_2 的状态值，即

$$v_{\pi_1}(s) \geq v_{\pi_2}, \quad \forall s \in \mathcal{S},$$

那么我们说 π_1 比 π_2 更好。而如果一个策略比所有其他策略都好，那么这个策略就是最优策略。

现在我们有几个问题:

1. 存在性：这样的最优策略存在么?
2. 唯一性：这样的最优策略唯一么?
3. 随机性：最优策略是唯一的还是随机的?
4. 算法：有什么算法能让我们找到最优策略和最优状态值?

3.2　贝尔曼最优方程

对于每个 $s \in \mathcal{S}$，贝尔曼最优方程的表达式如下：

$$\begin{aligned} v(s) &= \max_{\pi(s)\in\Pi(s)} \sum_{a\in\mathcal{A}} \pi(a \mid s) \left(\sum_{r\in\mathcal{R}} p(r \mid s,a) + \gamma \sum_{s'\in\mathcal{S}} \pi(s' \mid s,a) v(s') \right) \\ &= \max_{\pi(s)\in\Pi(s)} \sum_{a\in\mathcal{A}} \pi(a \mid s) q(s,a), \end{aligned}$$

其中

$$q(s,a) := \sum_{r\in\mathcal{R}} p(r \mid s,a) + \gamma \sum_{s'\in\mathcal{S}} \pi(s' \mid s,a) v(s').$$

这里 $v(s)$ 和 $v(s^{'})$ 是待解的未知量；$\pi(s)$ 表示状态 s 的策略；$\Pi(s)$ 是状态 s 所有可能策略的集合。

方程右侧的优化问题

例 3.2. 假设两个未知变量 $x, y \in \mathbb{R}$ 满足下列方程：

$$x = \max_{y \in \mathbb{R}} (2x - 1 - y^2).$$

这个方程乍一看比较奇怪：它包含两个未知数，并且右侧嵌套了一个优化问题。该方程的具体求解方法如下：首先求解方程右边的优化问题。右边是一个关于变量 y 的二次优化问题，很容易求解得出当 $y = 0$ 时，无论 x 取值为多少，目标函数取得最大值。第二步是求解 x，如果把 $y = 0$ 代入方程，那么方程变为 $x = 2x - 1$，此时很容易求解出 $x = 1$。因此，$y = 0$ 和 $x = 1$ 是该方程的解。

例 3.3. 给定 $q_1, q_2, q_3 \in \mathbb{R}$，我们希望找到 c_1, c_2, c_3 的最优值，从而求解下面的优化问题：

$$\max_{c_1, c_2, c_3} \sum_{i=1}^{3} c_i q_i$$

其中要求 $c_1 + c_2 + c_3 = 1$ 且 $c_1, c_2, c_3 \geq 0$。

求解这个问题的思路如下。首先 q_1, q_2, q_3 中一定存在一个最大值，不是一般性，假设 $q_3 \geq q_1, q_2$，那么最优解是 $c_3^* = 1, c_2^* = c_3^* = 0$。

收到上述例子启发，由于 $\sum \pi(a \mid s) = 1$, 我们有:

$$\sum_{a \in \mathcal{A}} \pi(a \mid s) q(s, a) \leq \sum_{a \in \mathcal{A}} \pi(a \mid s) \max_{a \in \mathcal{A}} q(s, a) = \max_{a \in \mathcal{A}} q(s, a),$$

即 $\sum_{a \in \mathcal{A}} \pi(a \mid s) q(s \mid a)$ 的最大值是 $\max_{a \in \mathcal{A}} q(s, a)$，而且等于最大值的条件是：

$$\pi(a \mid s) = \begin{cases} 1, & a = a^*, \\ 0, & a \neq a^*. \end{cases}$$

其中 $a^* = \arg\max_a q(s, a)$。因此，最优策略 $\pi(s)$ 应该选择具有最大 $q(s, a)$ 的动作。在解决了右侧从的优化问题之后，最优贝尔曼方程就变成了：

$$v(s) = \max_{a \in \mathcal{A}(s)} q(s, a).$$

矩阵—向量形式

假设有 n 个状态 $\{s_1, s_2, \cdots, s_n\}$，贝尔们最优方程的矩阵-向量形式为

$$v = \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v),$$

其中 $v = [v(s_1), v(s_2), \cdots, v(s_n)]^\top \in \mathbb{R}^n$ 是待解的未知量。由于 π 的最优值是由 v 决定的，上述式子的右侧实际上是 v 的函数，因此可以用一个函数 $f(v)$ 表示为

$$f(v) := \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v).$$

然后贝尔曼最优方程可以简洁地表达为:

$$v = f(v)$$

压缩映射定理*

方程右侧函数的压缩性质

定理 3.4 ($f(v)$ 的压缩性质)**.** *贝尔曼最优方程右侧函数 $f(v)$ 是一个压缩映射，即对任意的 $v_1, v_2 \in \mathbb{R}^{|\mathcal{S}|}$，有*

$$||f(v_1) - f(v_2)||_\infty \leq \gamma ||v_1 - v_2||_\infty,$$

其中 $\gamma \in (0, 1)$ 是折扣率，$||\cdot||_\infty$ 是向量所有元素中的最大绝对值。

证明. 考虑任意两个向量 $v_1, v_2 \in \mathbb{R}^{|\mathcal{S}|}$，假设 $\pi_1^* := \arg\max_\pi(r_\pi + \gamma P_\pi v_1)$ 和 $\pi_2^* := \arg\max_\pi(r_\pi + \gamma P_\pi v_2)$。那么:

$$f(v_1) = \max_\pi(r_\pi + \gamma P_\pi v_1) = r_{\pi_1^*} + \gamma P_{\pi_1^*} v_1 \geq r_{\pi_2^*} + \gamma P_{\pi_2^*} v_1,$$

$$f(v_2) = \max_\pi(r_\pi + \gamma P_\pi v_2) = r_{\pi_2^*} + \gamma P_{\pi_2^*} v_2 \geq r_{\pi_1^*} + \gamma P_{\pi_1^*} v_2,$$

其中"$\geq$" 是逐元素比较。因此:

$$\begin{aligned} f(v_1) - f(v_2) &= r_{\pi_1^*} + \gamma P_{\pi_1^*} v_1 - (r_{\pi_2^*} + \gamma P_{\pi_2^*} v_2) \\ &\leq r_{\pi_1^*} + \gamma P_{\pi_1^*} v_1 - (r_{\pi_1^*} + \gamma P_{\pi_1^*} v_2) \\ &= \gamma P_{\pi_1^*}(v_1 - v_2) \end{aligned}$$

类似的，可以证明 $f(v_2) - f(v_1) \leq \gamma P_{\pi_2^*}(v_2 - v_1)$。因此，

$$\gamma P_{\pi_2^*}(v_1 - v_2) \leq f(v_1) - f(v_2) \leq \gamma P_{\pi_1^*}(v_1 - v_2).$$

定义

$$z := \max\{|\gamma P_{\pi_2^*}(v_1 - v_2)|, |\gamma P_{\pi_1^*}(v_1 - v_2)|\} \in \mathbb{R}^{|\mathcal{S}|},$$

其中 $\max(\cdot)$ 和 $|\cdot|$ 是逐点操作元素。于是上述不等式可以化简为

$$|f(v_1) - f(v_2)| \leq z.$$

从而可以推出

$$||f(v_1) - f(v_2)||_\infty \leq ||z||_\infty,$$

另一方面，假设 z_i 是 z 的第 i 个元素，$p_i^\top$ 和 $q_i^\top$ 分别表示 $P_{\pi_1^*}$ 和 $P_{\pi_2^*}$ 的第 i 行，那么有

$$z_i = \max\{\gamma|p_i^\top(v_1 - v_2)|, \gamma|q_i^\top(v_1 - v_2)|\}$$

由于 p_i 中所有元素大于或等于 0 且所有元素之和等于 1，因此可以得到

$$|p_i^\top(v_1 - v_2)| \leq p_i^\top|v_1 - v_2| \leq ||v_1 - v_2||_\infty.$$

同理可得 $|q_i^\top(v_1 - v_2)| \leq ||v_1 - v_2||_\infty$。因此，我们有 $z_i \leq \gamma||v_1 - v_2||_\infty$，进而可得

$$||z||_\infty = \max|z_i| \leq \gamma||v_1 - v_2||_\infty.$$

结合上述式子

$$||f(v_1) - f(v_2)||_\infty \leq \gamma||v_1 - v_2||_\infty.$$

至此就完成了对 $f(v)$ 压缩性质的证明。 □

3.2.1 从贝尔曼最优方程到最优策略

有了前面的充分准备，下面求解贝尔曼最优方程，从而得到最优状态值 v^* 和最优策略 π^*。

1. 求解最优状态值 v^*：如果 v^* 是贝尔曼最优方程的解，那么它满足

$$v^* = f(v^*) = \max_{\pi\in\Pi}(r_\pi + \gamma P_\pi v^*.)$$

显然，v^* 是一个不动点。根据压缩映射原理，有如下重要结论。

定理 3.5 (存在性，唯一性，算法)**.** *贝尔曼最优方程 $v = f(v) = \max_{\pi\in\Pi}(r_\pi + \gamma P_\pi v)$ 始终存在唯一解 v^*，该解可以通过如下迭代算法求解：*

$$v_{k+1} = f(v_k) = \max_{\pi\in\Pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, \cdots$$

对任意给定的 v_0，当 $k \to \infty$，v_k 以指数速度收敛至 v^*

2. 求解最优策略 π^*：一旦得到了 v^* 的值，就可以通过下式求解得到一个最优策略：
$$\pi^* = \arg\max_{\pi\in\Pi}(r_\pi + \gamma P_\pi v^*)$$
将上式代入贝尔曼最优方程中可得
$$v^* = r_{\pi^*} + \gamma P_{\pi^*} v^*.$$
因此，$v^* = v_{\pi^*}$ 是策略 π^* 的状态值。从上式可以看出，贝尔曼最优方程是一个特殊的贝尔曼方程，其对应的策略是 π^*。

虽然上面提到了 v^* 是最优值、π^* 是最优策略，但是仍然没有证明他们的最优性，只是说明了 v^* 和 π^* 是贝尔曼最优方程的解而已。下面的定理证明了贝尔曼最优方程的解的最优性。

定理 3.6 (v^* 和 π^* 的最优性)**.** *如果 v^* 和 π^* 是贝尔曼最优方程的解，那么 v^* 是最优状态值，π^* 是最优策略，即对任意策略 π 都有*
$$v^* = v_{\pi^*} \geq v_\pi,$$
其中 v_π 是策略 π 的状态值，"$\geq$"是逐元素比较。

证明. 假设 π 是任意的一个策略，其贝尔曼方程为
$$v_\pi = r_\pi + \gamma P_\pi v_\pi.$$
由于
$$v^* = \max_\pi(r_\pi + \gamma P_\pi v^*) = r_{\pi^*} + \gamma P_{\pi^*} v^* \geq r_\pi + \gamma P_\pi v^*,$$
因此有
$$\begin{aligned} v^* - v_\pi &\geq (r_\pi + \gamma P_\pi v^*) - (r_\pi + \gamma P_\pi v_\pi) \\ &= \gamma P_\pi (v^* - v_\pi). \end{aligned}$$
反复应用上述不等式可得
$$v^* - v_\pi \geq \gamma P_\pi (v^* - v_\pi) \geq \gamma^2 P_\pi^2 (v^* - v_\pi) \geq \cdots \geq \gamma^n P_\pi^n (v^* - v_\pi).$$
由此可以推出
$$v^* - v_\pi \geq \lim_{n\to\infty} \gamma^n P_\pi^n (v^* - v_\pi) = 0.$$
最优 $v^* \geq v_\pi$ 对任何策略 π 都成立，所以 π^* 是最优策略且 v^* 是最优状态值。 □

虽然上面反复提到了最优策略 π^*，但是它长什么样子呢？它是确定性策略还是随机性策略呢？下面的定理将回答这个问题。

定理 3.7 (贪婪最优策略)**.** 假设 v^* 是贝尔曼最优方程的最优状态值解，那么下面的确定性贪婪策略是一个最优策略：

$$\pi^*(a \mid s) = \begin{cases} 1, & a = a^*(s), \\ 0, & a \neq a^*(s), \end{cases} \quad s \in \mathcal{S},$$

其中

$$a^*(s) = \arg\max_a q^*(a, s),$$

且

$$q^*(s, a) := \sum_{r \in \mathcal{R}} p(r \mid a, s) r + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) v^*(s').$$

证明. 最优策略的矩阵-向量形式是 $\pi^* = \arg\max_\pi (r_\pi + \gamma P_\pi v^*)$，按元素展开的形式为

$$\pi^* = \arg\max_{\pi \in \Pi} \sum_{a \in \mathcal{A}} \pi(a \mid s) \underbrace{\left(\sum_{r \in \mathcal{R}} p(r \mid s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) v^*(s') \right)}_{q^*(s,a)}, \quad s \in \mathcal{S}.$$

为了最大化 $\sum_{a \in \mathcal{A}} \pi(a \mid s) q^*(s, a)$，策略 $\pi(s)$ 应该选择最大 $q^*(s, a)$ 动作，这是因为

$$\sum_{a \in \mathcal{A}} \pi(a \mid s) q^*(s, a) \leq \sum_{a \in \mathcal{A}} \pi(a \mid s) \max_{a \in \mathcal{A}} q^*(s, a) = \max_{a \in \mathcal{A}} q^*(s, a).$$

□

上述定理指出总是存在一个确定性的贪婪策略是最优的。最后我们指出两个最优策略 π^* 的两个重要性质。

1. 最优策略的唯一性：尽管最优状态值 v^* 是唯一的，但对应的最优策略可能不唯一。

2. 最优策略的随机性：最优策略可以是随机的或确定的，然而根据上述定理，可以肯定地说，总是存在一个确定性的最优策略。

第四章　值迭代与策略迭代

4.1　值迭代算法

该算法实际上是压缩映射定理解决贝尔曼最优方程的算法。为了方便阅读，我们将该算法再次写出来：

$$v_{k+1} = \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0, 1, 2, \cdots$$

随着 k 趋向无穷大，v_k 和 π_k 分别收敛于最优状态值和最优策略。

该迭代算法的每次迭代包含两个步骤。

1. 第一步是策略更新。在数学上，它旨在找到一个能解决以下优化问题的策略：

   $$\pi_{k+1} = \arg\max_\pi(r_\pi + \gamma P_\pi v_k),$$

   其中 v_k 是上一次迭代得到的值。

2. 第二步是值更新。在数学上，它通过下式来计算新的值 v_{k+1}：

   $$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

   其中 v_{k+1} 将用于下一次迭代。

展开形式和实现细节

在第 k 次迭代中，状态 s 对应的策略更新和值更新的步骤的细节如下所示。

1. 策略更新：$\pi_{k+1} = \arg\max_\pi(r_\pi + \gamma P_\pi v_k)$ 的按元素展开形式为

   $$\pi_{k+1}(s) = \arg\max_\pi \sum_a \pi(a \mid s) \underbrace{\left(\sum_r p(r \mid s, a) r + \gamma \sum_{s'} p(s' \mid s, a) v_k(s')\right)}_{q_k(s,a)}, \quad s \in \mathcal{S}.$$

上述优化问题的最优解为

$$\pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s), \\ 0, & a \neq a_k^*(s), \end{cases}$$

其中 $a^*(s) = \arg\max_a q_k(a, s)$。此外，如果有多个动作的值相同并且最大，那么可以选择其中任意一个动作。

2. 值更新：$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$ 的按元素展开形式是

$$v_{k+1}(s) = \sum_a \pi_{k+1}(a \mid s) \underbrace{\left(\sum_r p(r \mid s, a) r + \gamma \sum_{s'} p(s' \mid s, a) v_k(s') \right)}_{q_k(s,a)}, \quad s \in \mathcal{S}.$$

将 $\pi_{k+1}(a \mid s)$ 代入上式可得

$$v_{k+1}(s) = \max_a q_k(s, a)$$

即新的值等于状态 s 对应的最大 q 值。

上面两个步骤可以概述为如下形式：

$v_k(s) \rightarrow$ **计算** $q_k(s,a) \rightarrow$ **计算新策略** $\pi_{k+1}(s) \rightarrow$ **计算新值** $v_{k+1}(s) = \max_a q_k(s,a)$

注意，在上述迭代的过程中 v_k 不一定就是状态值了。

算法 4.1：值迭代算法

初始化： 已知模型，即任意 (s,a) 对应的 $p(r|s,a)$ 和 $p(s'|s,a)$。初始值 v_0。

目标： 求解最优状态值和最优策略。

当 v_k 尚未收敛时（例如 $\|v_k - v_{k-1}\|$ 大于给定阈值时），进行如下迭代

　对每个状态 $s \in \mathcal{S}$

　　对每个动作 $a \in \mathcal{A}(s)$

　　计算 q 值：$q_k(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_k(s')$

　　最大价值动作：$a_k^*(s) = \arg\max_a q_k(s,a)$

　　策略更新：$\pi_{k+1}(a|s) = 1$ 如果 $a = a_k^*$；否则 $\pi_{k+1}(a|s) = 0$

　　值更新：$v_{k+1}(s) = \max_a q_k(s,a)$

图 4.1: 值迭代算法

4.2　策略迭代算法

策略迭代不想值迭代那样直接求解贝尔曼最优方程，而是在策略评价和策略改进之间来回切换，**这种思想在强化学习算法中广泛存在**。

4.2.1　算法概述

策略迭代也是一种迭代算法，每次迭代包含两个步骤。具体来说，第 k 次迭代的两个步骤如下所述。

1. 第一步是策略评价。顾名思义，这个步骤是用来评估上一次迭代得到的策略 π_k。从数学上来说，该步骤就是求解下面的贝尔曼方程，从而得到 π_k 的状态值：
$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$
其中 π_k 是上一次迭代中得到的策略，v_{π_k} 是该策略对应的状态值，r_{π_k} 和 P_{π_k} 根据系统模型得到。

2. 第二步是策略改进。顾名思义，这个步骤是用来改进策略从而得到更好的新策略。从数学上来说，在第一步得到 v_{π_k} 后，此步骤会利用下式得到新的策略 π_{k+1}：

为了透彻理解该算法，我们需要回答下面三个问题：

1. 在策略评估步骤中，如何计算状态值 v_{π_k}？
2. 在策略改进步骤中，新策略 π_{k+1} 为什么比 π_k 好？
3. 为什么这个算法最终能够收敛到最优策略？

在策略评估步骤中，如何计算状态值 v_{π_k}？

通常采用数值迭代算法计算状态值：

$$v_{\pi_k}^{(j+1)} = r_\pi + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \cdots$$

在理论上，该算法需要迭代无限步才能收敛到真实的状态值 v_{π_k}。但是在实际中，算法不可能执行无限步，我们让算法在满足某个条件后终止迭代，例如 $||v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}||$ 小于一个数值或者 j 大于一个数值。值得注意的是，虽然我们在策略迭代过程中每一步状态值都只是近似，但是最终我们仍然能找到最优策略。

在策略改进步骤中，新策略 π_{k+1} 为什么比 π_k 好？

引理 4.1 (策略改进)**.** 如果 $\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k})$，那么 $v_{\pi_{k+1}} \geq v_{\pi_k}$。

证明. 由于 $v_{\pi_{k+1}}$ 和 v_{π_k} 都是状态值，它们分别满足下面的贝尔曼方程:

$$v_{\pi_{k+1}} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_{k+1}},$$

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

由于 $\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k})$，因此

$$r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_k} \geq r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}.$$

因此

$$\begin{aligned} v_{\pi_k} - v_{\pi_{k+1}} &= (r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}) - (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_{k+1}}) \\ &\leq (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_k}) - (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_{k+1}}) \\ &\leq \gamma P_{\pi_{k+1}} (v_{\pi_k} - v_{\pi_{k+1}}) \end{aligned}$$

反复应用上式

$$\begin{aligned} v_{\pi_k} - v_{\pi_{k+1}} \leq \gamma^2 P^2_{\pi_{k+1}} (v_{\pi_k} - v_{\pi_{k+1}}) \leq \cdots \leq \gamma^n P^n_{\pi_{k+1}} (v_{\pi_k} - v_{\pi_{k+1}}) \\ \leq \lim_{n\to\infty} \gamma^n P^n_{\pi_{k+1}} (v_{\pi_k} - v_{\pi_{k+1}}) = 0 \end{aligned}$$

于是 $v_{\pi_k} - v_{\pi_{k+1}} \leq 0$ 说明了 π_{k+1} 比 π_k 更优。 □

为什么这个算法最终能够收敛到最优策略？

策略迭代算法会生成两个序列。第一个是策略序列 $\{\pi_0, \pi_1, \cdots, \pi_k, \cdots\}$，第二个是状态值序列 $\{v_{\pi_0}, v_{\pi_1}, \cdots, v_{\pi_k}, \cdots\}$。假设 v^* 是最优状态值，那么对于所有 k 都有 $v_{\pi_k} \leq v^*$。那么

$$v_{\pi_0} \leq v_{\pi_1} \leq v_{\pi_2} \leq \cdots \leq v_{\pi_k} \leq \cdots \leq v^*$$

由于 v_{π_k} 单调递增且有上界，根据单调收敛定理，当 k 趋近无穷大时，v_{π_k} 会收敛到一个常数，记为 v_∞。那么，收敛值 v_∞ 是否等于最优值 v^* 呢? 下面这个定理回答了这个问题。

定理 4.2 (策略迭代的收敛性)**.** 由策略迭代算法生成的状态值序列 $\{v_{\pi_k}\}_{k=0}^\infty$ 收敛于最优状态值 v^*。因此，策略序列 $\{\pi_k\}_k^\infty$ 收敛于一个最优策略。

证明. 为了证明 $\{v_{\pi_k}\}$ 的收敛性，我们引入另外一个序列 $\{v_k\}$，该序列由下面的迭代算法产生：

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k).$$

这个迭代算法实际上就是值迭代算法。我们已经知道任意的初始值 v_0，v_k 都会收敛到 v^*。

对于任意的 π_0，总是能找到 v_0 使得 $v_0 \leq v_{\pi_0}$。下面用递归法证明 $v_k \leq v_{\pi_k} \leq v^*$ 对任意 $k \geq 1$ 都成立。

假设 $v_{\pi_k} \geq v_k$ 对 k 成立，那么对于 $k+1$ 可得

$$\begin{aligned}
v_{\pi_{k+1}} - v_{k+1} &= (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_{k+1}}) - \max_{\pi}(r_\pi + \gamma P_\pi v_k) \\
&\geq (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_k}) - \max_{\pi}(r_\pi + \gamma P_\pi v_k) \\
&= (r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_{\pi_k}) - (r_{\pi'} + \gamma P_{\pi'} v_k) \\
&\geq (r_{\pi'} + \gamma P_{\pi'} v_{\pi_k}) - (r_{\pi'} + \gamma P_{\pi'} v_k) \\
&= \gamma P_{\pi'}(v_{\pi_k} - v_k) \geq 0
\end{aligned}$$

根据递归可得：$v_k \leq v_{\pi_k} \leq v^*$ 对任意 $k \geq 1$ 都成立。其次根据夹逼法：

$$\lim_{k\to\infty} v_{\pi_k} = v^*.$$

□

4.2.2 算法的展开形式

1. 第一，策略评价步骤利用迭代算法来求解贝尔曼公式从而得到 v_{π_k}，其元素展开形式为

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a \mid s)\left(\sum_r p(r \mid s,a)r + \gamma \sum_{s'} p(s' \mid s,a) v_{\pi_k}^{(j)}(s')\right), \quad s \in \mathcal{S},$$

   其中 $j = 0, 1, 2, \cdots$

2. 第二，策略改进步骤时求解 $\pi_{k+1} = \arg\max_\pi(r_\pi + \gamma P_\pi v_{\pi_k})$，该式的元素展开式为

$$\pi_{k+1}(s) = \arg\max_\pi \sum_a \pi(a \mid s) \underbrace{\left(\sum_r p(r \mid s,a)r + \gamma \sum_{s'} p(s' \mid s,a) v_{\pi_k}(s')\right)}_{q_{\pi_k}(s,a}, \quad s \in \mathcal{S}.$$

其中 $q_{\pi_k}(a,s)$ 是策略 π_k 对应的动作值。令 $a^* = \arg\max_a q_{\pi_k}(a,s)$ 为最大动作值，那么上述优化问题的解是

$$\pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s), \\ 0, & a \neq a_k^*(s), \end{cases}$$

算法 4.2: 策略迭代算法

初始化：已知模型，即任意 (s,a) 对应的 $p(r|s,a)$ 和 $p(s'|s,a)$，初始策略 π_0。

目标：寻找最优状态值和最优策略。

当 v_{π_k} 未收敛时，进行第 k 次迭代

策略评价：

选择初始值 $v_{\pi_k}^{(0)}$

当 $v_{\pi_k}^{(j)}$ 未收敛时，进行第 j 次迭代

对每一个状态 $s \in \mathcal{S}$

$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s')\right]$

策略改进：

对每一个状态 $s \in \mathcal{S}$

对每一个动作 $a \in \mathcal{A}$

$q_{\pi_k}(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s')$

$a_k^*(s) = \arg\max_a q_{\pi_k}(s,a)$

$\pi_{k+1}(a|s) = 1$ 如果 $a = a_k^*$；否则 $\pi_{k+1}(a|s) = 0$

图 4.2: 策略迭代算法

4.3　截断策略迭代算法

对比之前的值迭代算法和策略迭代算法：如果两个算法从相同的初始

$$\begin{aligned} &\text{策略迭代算法：} \pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots \\ &\text{值迭代算法：} \qquad v_0 \xrightarrow{PU} \pi_1' \xrightarrow{VU} v_1 \xrightarrow{PU} \pi_2' \xrightarrow{VU} v_2 \xrightarrow{PU} \dots \end{aligned}$$

图 4.3: 值迭代和策略迭代对比

值 v_0 开始迭代，两者的主要区别在于——求解 π_k 对应状态值时迭代次数

不一样。值迭代只迭代一次，策略迭代迭代无限次。如果只迭代一次 (值迭代) 会导致首先速度变慢；如果迭代无限次 (策略迭代)，则每次计算的花费很高。为了平衡两者，提出一种折中的方法：每次求解 π_k 对应的状态值 v_k 时，只迭代有限次数 j，这便是截断策略迭代算法的思想核心。但其实策略迭代在实际应用过程中也使用了截断的思想，因为现实中我们无法进行无限次迭代。截断策略迭代的伪代码：

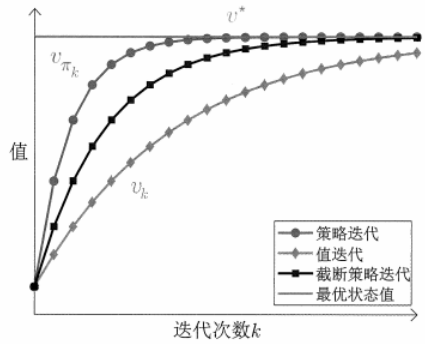

图 4.5　值迭代、策略迭代、截断策略迭代算法收敛速度示意图。

图 4.4: 三种算法的收敛速度比较

算法 4.3：截断策略迭代算法

初始化： 已知模型，即任意 (s,a) 对应的 $p(r|s,a)$ 和 $p(s'|s,a)$。初始策略 π_0。

目标： 寻找最优状态值和最优策略。

当 v_k 未收敛时，进行第 k 次迭代

策略评价：

选择初始值为 $v_k^{(0)} = v_{k-1}$。设置最大迭代次数为 j_{truncate}。

当 $j < j_{\text{truncate}}$ 时

对每一个状态 $s \in \mathcal{S}$

$$v_k^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_k^{(j)}(s')\right]$$

令 $v_k = v_k^{(j_{\text{truncate}})}$

策略改进：

对每一个状态 $s \in \mathcal{S}$

对每一个动作 $a \in \mathcal{A}(s)$

$$q_k(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_k(s')$$

$a_k^*(s) = \arg\max_a q_k(s,a)$

$\pi_{k+1}(a|s) = 1$ 如果 $a = a_k^*$；否则 $\pi_{k+1}(a|s) = 0$

图 4.5: 截断策略迭代算法

第五章　蒙特卡罗方法

上一张介绍了系统模型求解最优策略的方法，本章将介绍无需模型的强化学习算法。如何在没有模型的情况下找到最优策略？实际上，思路很简单：如果没有模型，则必须有数据；如果没有数据，则必须有模型；如果两者都没有，那么就无法找到最优策略。强化学习中，“数据”通常指的是智能体与环境交互的经验。

5.1　MC Basic: 最简单的基于蒙特卡罗的算法

将策略迭代算法转换为无需模型

蒙特卡罗方法是以策略迭代算法为基础的。策略迭代算法的每次迭代有两个步骤：第一步是策略评估，旨在通过求解贝尔曼方程 $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$ 以得到 v_{π_k}；第二布是策略改进，旨在计算贪婪策略 $\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k})$ 以得到更好的策略。具体来说，策略改进步骤的元素展开形式是

$$
\begin{aligned}
\pi_{k+1}(s) &= \arg\max_\pi \sum_a \pi(a \mid s) \left[\sum_r p(r \mid s,a) r + \gamma \sum_{s'} p(s' \mid s,a) v_{\pi_k}(s') \right] \\
&= \arg\max_\pi \sum_a \pi(a \mid s) q_{\pi_k}(s,a), \quad s \in \mathcal{S}
\end{aligned}
$$

从上式能看出，动作值 $q_{\pi_k}(s,a)$ 是策略迭代算法的核心：第一步策略评估就是再计算状态值进而得到动作值；第二部策略改进就是选取动作值最大的动作作为新策略。

在明白动作值得的核心作用之后，让我们重新审视计算动作值得方法，实际上有两种。

1. 第一，基于模型的方法。首先求解贝尔曼方程得到状态值 v_{π_k}，然后基于下式得到动作值 $q_{\pi_k}(s,a)$：

$$
q_{\pi_k}(s,a) = \sum_r p(r \mid s,a) r + \gamma \sum_{s'} p(s' \mid s,a) v_{\pi_k}(s').
$$

这种方法需要知道模型 $\{p(r \mid s,a),\, p(s^{'} \mid s,a)\}$。

2. 第二，无需模型的方法。让我们回忆一下动作价值的原始定义：

$$\begin{aligned} q_{\pi_k}(s,a) &= \mathbb{E}[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s,\, A_t = a], \end{aligned}$$

值得注意的是，因为 $q_{\pi_k}(a,s)$ 是一个期望值，所以可以通过蒙特卡罗方法数据来估计。具体操作如下：从特定的状态 s 开始，强制执行动作 a 之后，智能体完全按照当前策略 π_k 与环境交互，直到这一个 Episode 结束。记录下这个 Episode 中获得的所有奖励 $R_{t+1}, R_{t+2}, \cdots$，并根据折扣因子 γ 计算总回报 $g_{\pi_k}(s,a)$：

$$g_{\pi_k}(s,a) = \sum_{i=1}^{\infty} \gamma^{i-1} R_{t+i}$$

重复上述过程 n 次，得到 n 个独立的回报样本：$g_{\pi_k}^{(1)}, g_{\pi_k}^{(2)}, \cdots, g_{\pi_k}^{(n)}$。那么，这些回报的平均值可以用来近似 $q_{\pi_k}(s,a)$，即

$$q_{\pi_k}(s,a) = \mathbb{E}[G_t \mid S_t = s,\, A_t = a] \approx \frac{1}{n} \sum_{i=1}^{n} g_{\pi_k}^{(i)}(s,a).$$

根据大数定律，如果 n 足够大，上面的近似会足够精确。

基于蒙特卡罗的强化学习的基本思想就是使用上式来估计动作值，从而替代策略迭代算法中需要模型的模块。

MC Basic 算法

从初始策略 π_0 开始，该算法在第 k 次迭代中有两个步骤。

1. 步骤一：策略评估。这一步用于估算所有 (s,a) 的 $q_{\pi_k}(s,a)$。具体来说，对于每个 (s,a)，收集足够多的回合进而求其回报的平均值来近似 $q_{\pi_k}(s,a)$。

2. 步骤二：策略改进。这一步通过 $\pi_{k+1} = \arg\max_{\pi} \sum_a \pi(a \mid s) q_{\pi_k}(s,a)$ 得到所有 $s \in \mathcal{S}$ 的新策略，即 $\pi_{k+1}(a_k^* \mid s) = 1$，其中 $a_k^* = \arg\max_a q_k(s,a)$。

在 MC Basic 算法中我们不需要显示的计算 $v_k(s)$，而 $v_k(s)$ 只是计算 $q_k(s,a)$ 后顺带求出来的结果：

$$v_k(s) = \sum_a \pi(a \mid s) q_k(a,s).$$

有两个重要超参数会极大程度上影响 MC Basic 算法的效率：回合长度和

算法 5.1：MC Basic（策略迭代算法的无模型版本）

初始化: 初始策略 π_0

目标: 寻找最优策略。

对于第 k 次迭代（$k = 0, 1, 2, \ldots$）

　对于每个状态 $s \in \mathcal{S}$

　　对于每个动作 $a \in \mathcal{A}(s)$

　　执行 π_k 收集从 (s, a) 开始的足够多的回合

　　策略评估：

　　$q_{\pi_k}(s, a) \approx q_k(s, a) =$ 所有从 (s, a) 开始的回合的回报的平均值

　　策略改进：

　　$a_k^*(s) = \arg\max_a q_k(s, a)$

　　如果 $a = a_k^*$，则 $\pi_{k+1}(a|s) = 1$；否则 $\pi_{k+1}(a|s) = 0$

图 5.1: MC Basic 算法

稀疏奖励。如果设置的回合长度过短，用 MC Basic 算法得到的策略和价值都不是最优的。

上述分析涉及到一个重要的奖励设计问题：稀疏奖励。稀疏奖励指的是除非达到目标状态，否则无法获得任何正奖励。稀疏奖励要求回合必须达到目标。当状态空间比较大或者系统随机性比较强时，一个回合达到目标的概率是比较低的。因此，稀疏奖励降低了学习效率。解决这个问题的一个简单方法是设计非稀疏奖励或者稠密奖励。

5.2　MC Exploring Starts 算法

5.2.1　更高效地利用样本

假设我们通过执行策略 π 获得了一系列样本：

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \cdots$$

上式中的 s 和 a 的下标指的是状态和动作的索引，而不是时间步数。如果一个状态-动作配对在一个回合中出现了一次，那么我们称该状态-动作被访问一次。有多种方法来利用这些访问。

第一种也是最简单的方法是：一个回合仅用于估计该回合最开始访问的状态-动作值的价值。例如上式，这个回合最开始访问的是 (s_1, a_2)，那么这个回合只是被用来估计 (s_1, a_2) 的动作价值。前面介绍的 MC Basic 算法

就是基于这种方法的。

虽然这种方法简单，但是它没有充分利用样本，因为回合还访问了许多其他的状态-动作，例如 $(s_2, a_4), (s_2, a_3), (s_5, a_1)$。实际上，一个回合可以被用于估计其他状态-动作价值。例如，我们可以将上述式子中的回合分解成多个子回合：如上所示，一个长的回合可以被看成很多个从不同状态-动作

$$
\begin{aligned}
s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \quad & [\text{原始回合}] \\
s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \quad & [\text{从 } (s_2, a_4) \text{ 开始的子回合}] \\
s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \quad & [\text{从 } (s_1, a_2) \text{ 开始的子回合}] \\
s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \quad & [\text{从 } (s_2, a_3) \text{ 开始的子回合}] \\
s_5 \xrightarrow{a_1} \dots \quad & [\text{从 } (s_5, a_1) \text{ 开始的子回合}]
\end{aligned}
$$

图 5.2: 子回合

出发的子回合，因此可以估计多个不同状态-动作的价值。通过这种方法，一个回合中的样本可以被更有效地利用。

在一个回合中，一个状态-动作可能会被多次访问。例如，在长回合中，(s_1, a_2) 被访问了两次。如果我们只考虑第一次访问，这种方法被称为首次访问。例如，以第一次出现的 (s_1, a_2) 为开始地回合会被用来估计 (s_1, a_2) 的动作值，如果我们考虑每一次访问，则这种方法被称为每次访问。例如，每次出现 (s_1, a_2) 时，以其未开的子回合会被用来估计其动作值。

在样本使用效率方面，每次访问都利用的方法效率是最高的。如果一个回合最够长，以至于它可以多次访问所有状态-动作值，那么这一个回合就足以估计所有的状态-动作价值，此时则需要使用每次访问的方法。然而，通过这种方法获得的样本回报是相关的 (这会影响蒙塔卡罗模拟，因为大数定律的要求是**独立**同分布的样本。)，因为从第二次访问开始的轨迹只是从第一次访问开始的轨迹的一部分。尽管如此，如果两次访问在轨迹中相隔很远，那么相关性也不会很强。

5.2.2　更高效地更新策略

除了上一节介绍的高效利用样本，我们还可以更高效地更新策略。有两种更新策略的方法。

1. 第一种方法：在策略评价步骤中，收集从某一个状态-动作开始的所有回合，然后使用所有回合的平均回报来近似动作值，进而再更新策略。

这种方法已经在 MC Basic 算法中被采用。它的一个缺点是智能体必须等到所有回合都被收集完毕之后才能估计动作值。

2. 第二种方法：在策略评价步骤中，收集从某一状态-动作开始的单个回合，然后使用这个单个回合的回报来近似动作值，进而再立即更新策略。这样的好处是不需要等到收集完所有回合，而是可以在得到一个回合后立即更新值和策略。

这个有点像从梯度下降发展出随机梯度下降的想法。虽然第二种方法对动作值的近似不够准确，但是能加快训练速度并且保证策略收敛到最优策略。

5.2.3 算法描述

算法 5.2：MC Exploring Starts 算法（MC Basic 算法的推广）

初始化： 初始策略 $\pi_0(a|s)$，所有 (s,a) 的初始价值 $q(s,a)$。$\text{Return}(s,a) = 0$ 和 $\text{Number}(s,a) = 0$ 适用于所有 (s,a)。

目标： 寻找一个最优策略。

对每个回合

　回合生成：选择一个起始状态-动作 (s_0, a_0)（确保所有状态-动作都可能被选中，这就是 Exploring Starts 条件）。按照当前策略，生成一个长度为 T 的回合：$s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$。

　每个回合的初始化：$g \leftarrow 0$

　对回合中的每一步 $t = T-1, T-2, \ldots, 0$

　　$g \leftarrow \gamma g + r_{t+1}$

　　$\text{Return}(s_t, a_t) \leftarrow \text{Return}(s_t, a_t) + g$

　　$\text{Number}(s_t, a_t) \leftarrow \text{Number}(s_t, a_t) + 1$

　　策略评价：

　　$q(s_t, a_t) \leftarrow \text{Return}(s_t, a_t)/\text{Number}(s_t, a_t)$

　　策略改进：

　　对 $a = \arg\max_a q(s_t, a)$，$\pi(a|s_t) = 1$；对其他 a，$\pi(a|s_t) = 0$

图 5.3: MC Exploring Starts 算法

注意，该算法使用了每次访问，并且用另一个技巧来提高效率：在计算从每个状态-动作开始获得的回报时，采用“回溯”的方式，即先从回合最后的状态-动作开始，慢慢推回最初的状态-动作，这样可以使算法更高效。

MC Exploring Starts 算法有一个条件，即 Exploring Starts 条件：对每一个状态-动作，都要有足够多的回合从它出发 (这里“每一个”对应了

Exploring，“出发” 对应了 Starts)。只有这样，我们才能准确估计每一个状态-动作值，进而成功找到最优策略。否则，如果存在一个动作-状态，所有的回合都没有从它出发，那么我们无法估计出该状态-动作的价值。即使这个动作确实是最优的，凡是因为它没有被访问过，所以可能刚好被错过。

MC Basic 和 MC Exploring Starts 都需要 Exploring Starts 这个条件。然而，这个条件在许多应用中很难满足，因为我们难以确保有足够多的回合从每一个状态-动作出发。实际上，如果只是为了确保每一个状态-动作都被访问到，那么可以用下面介绍的软策略的方法。

5.3　MC ϵ-Greedy 算法

下面进一步推广 MC Exploring Starts，使其不再依赖于 Exploring Starts 这个不合理条件。为此，我们需要引入软策略。如果一个策略能在任意状态下有非零概率选择任意动作, 那么该策略被称为软策略。给定一个软策略，即使只有一个回合，只要这个回合足够长，它就会多次访问每个状态-动作值。此时，我们不再需要从不同状态-动作出发的很多回合，而 Exploring Starts 这个条件就可以避免了。

5.3.1　ϵ-Greedy 策略

一种常见的软策略是 ϵ-Greedy 策略。具体来说，假设 $\epsilon \in [0,1]$。相应的 ϵ-Greedy 策略具有以下形式:

$$\pi(a \mid s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1) & \text{对于最大值动作} \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{对于其他 } |\mathcal{A}(s)| - 1 \text{ 个动作} \end{cases}$$

其中 $|\mathcal{A}(s)|$ 表示于 s 相关联的动作数量。从上式可以看出，ϵ-Greedy 是随机性策略，它选择具有最大价值的动作的概率最高，而选择其他所有动作的概率都相同。

当 $\epsilon = 0$ 时，ϵ-Greedy 变为普通的贪婪策略，此时策略的搜索性最弱，因为只会选择最大价值动作。当 $\epsilon = 1$ 时，所有动作被选择的概率都等一 $1/|\mathcal{A}(s)|$，此时策略的探索性最强。

由于 ϵ-Greedy 策略是随机的，那么如何根据这样的策略选择动作呢? 我们可以首先按照均匀分布在 $[0,1]$ 生成一个随机数 x。如果 $x \geq \epsilon$，那么选择最大价值动作；如果 $x < \epsilon$，那么按照相同的概率 $\frac{1}{|\mathcal{A}(s)|}$ 选择任意一个动作 (**此时可能再次选到最大价值动作**)。通过这种方式，选到最大价值动作的总概率是 $1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}$，而选择任意其他动作的概率是 $\frac{\epsilon}{|\mathcal{A}(s)|}$。

5.3.2 算法描述

下面将 ϵ-Greedy 策略融合到 MC Exploring Starts 算法中，可以得到新的 MC ϵ-Greedy 算法，该算法已经不再依赖于 Exploring Starts 的条件。

为此，我们需要将 MC Exploring Starts 算法中的贪婪策略改为 ϵ-Greedy 策略。具体来说，在 MC Exploring Starts 算法中，策略改进的步骤是选择如下的 Greedy 策略：

$$\pi_{k+1}(s) = \arg\max_{\pi \in \Pi} \sum_a \pi(a \mid s) q_{\pi_k}(s, a),$$

其中 Π 表示所有可能策略的集合。我们知道上述式子的最优解是一个贪婪策略:

$$\pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*, \\ 0, & a \neq a_k^*, \end{cases}$$

其中 $a_k^* = \arg\max_a q_{\pi_k}(s, a)$。

现在，策略改进步骤需要改成

$$\pi_{k+1}(s) = \arg\max_{\pi \in \Pi_\epsilon} \sum_a \pi(a \mid s) q_{\pi_k}(s, a),$$

其中 Π_ϵ 表示所有 ϵ-Greedy 策略的集合，这里 ϵ 值是给定的。通过这种方式，我们强制把策略限制为 ϵ-Greedy。上述方程的解不难得到：

$$\pi_{k+1}(a \mid s) = \begin{cases} 1 - \frac{|\mathcal{A}(s)|-1}{|\mathcal{A}(s)|}\epsilon, & a = a_k^*, \\ \frac{1}{|\mathcal{A}(s)|}\epsilon, & a \neq a_k^*, \end{cases}$$

其中 $a_k^* = \arg\max_a q_{\pi_k}(s, a)$。

基于上述改变，我们得到了另一种称为 MC ϵ-Greedy 的算法。由于 ϵ-Greedy 策略具有一定的探索性，这样就不需要有多个回合从每一个状态-动作出发了。如果策略改进步骤中将 Greedy 策略限制为 ϵ-Greedy 策略，我们不一定能保证获得最优策略。当有足够多的样本时，算法总是能收敛到 Π_ϵ 中最优的策略。不过该策略仅在 Π_ϵ 中是最优的，而在 Π 中不一定。所以这个算法只是增加了策略的探索性，而牺牲了策略的最优性。不过，如果 ϵ 足够小，那么 Π_ϵ 中的最优策略与 Π 中的最优策略是非常接近的。

算法 5.3：MC ϵ-Greedy 算法（MC Exploring Starts 算法的推广）

初始化: 初始策略 $\pi_0(a|s)$，所有 (s,a) 的初始值 $q(s,a)$。$\text{Return}(s,a) = 0$ 和 $\text{Number}(s,a) = 0$ 对于所有 (s,a)。$\epsilon \in (0,1]$

目标: 寻找最优策略。

对每个回合

　回合生成：选择一个初始状态-动作 (s_0, a_0)（不需要每一个状态-动作都被选到）。执行当前策略，生成长度为 T 的回合：$s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$。

　每个回合的初始化：$g \leftarrow 0$

　对回合的每一步 $t = T-1, T-2, \ldots, 0$

　　$g \leftarrow \gamma g + r_{t+1}$

　　$\text{Return}(s_t, a_t) \leftarrow \text{Return}(s_t, a_t) + g$

　　$\text{Number}(s_t, a_t) \leftarrow \text{Number}(s_t, a_t) + 1$

　　策略评估：

　　$q(s_t, a_t) \leftarrow \text{Return}(s_t, a_t)/\text{Number}(s_t, a_t)$

　　策略改进：

　　如果 $a^* = \arg\max_a q(s_t, a)$，那么

$$\pi(a|s_t) = \begin{cases} 1 - \frac{|\mathcal{A}(s_t)|-1}{|\mathcal{A}(s_t)|}\epsilon, & a = a^* \\ \frac{1}{|\mathcal{A}(s_t)|}\epsilon, & a \neq a^* \end{cases}$$

图 5.4: e-贪婪算法

第六章　随机近似算法

考虑一个随机变量 X，它取值于有限集 $\mathcal{X}$。我们的目标是估计期望值 $\mathbb{E}[X]$。假设有一个独立同分布的样本序列 $\{x_i\}_{i=1}^n$，那么 X 的期望值可以通过这些样本的平均值来近似：

$$\mathbb{E}[X] \approx \overline{x} := \frac{1}{n}\sum_{i=1}^{n} x_i$$

有两种方法可以计算平均值 $\overline{x}$。第一种是非增量式的：首先搜集所有的样本，然后计算平均值。这种方法的缺点是如果样本数量很大，可能需要等待很久才能把所有样本收集完毕。第二种是增量式的。具体来说，令

$$w_{k+1} := \frac{1}{k}\sum_{i=1}^{k} x_i, \quad k = 1, 2, \cdots$$

类似地

$$w_k = \frac{1}{k-1}\sum_{i=1}^{k-1} x_i, \quad k = 1, 2, \cdots$$

实际上，w_{k+1} 可以用 w_k 表示

$$w_{k+1} = \frac{1}{k}\sum_{i=1}^{k} x_i = \frac{1}{k}\left(\sum_{i=1}^{k-1} x_i + x_k\right) = \frac{1}{k}((k-1)w_k + x_k) = w_k - \frac{1}{k}(w_k - x_k).$$

因此我们得到增量式算法:

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k).$$

上述算法的优势在于，每一次接收到一个样本时，平均值就可以立即计算出来，进而近似 $\mathbb{E}[X]$。由于样本不足，开始时近似可能不够准确，不过这总比没有好。随着获得更多样本，估计的准确度会逐渐提高。

最后，我们可以把上述公式推广到一个更一般化的算法:

$$w_{k+1} = w_k - \alpha_k(w_k - x_k).$$

6.0.1 在期望值估计问中的应用

下面展示的算法实际上是一个特殊的 RM 算法，因此其收敛性也可以得到保证。具体来说，如意如下函数:

$$g(w) := w - \mathbb{E}[X]$$

此时，求解期望值 $\mathbb{E}[X]$ 的问题可以转换成求解方程 $g(w) = 0$ 的问题。由于我们能得到 X 的随机样本 x，对任意给定的 w 值，我们能获得的带噪声的观测值为 $\overline{g} := w - x$，$\overline{g}$ 可以进一步写成

$$\begin{aligned}\overline{g}(w,\eta) &= w - x \\ &= (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) := g(w) + \eta\end{aligned}$$

其中 $\eta = \mathbb{E}[X] - x$。这样我们就把期望估计的问题转换为一个可以用 RM 算法求解的问题。相应的 RM 算法是:

$$w_{k+1} = w_k - \alpha_k \overline{g}(w,\eta) = w_k - \alpha(w_k - x_k)$$

根据 RM 定理，如果 $\sum_{k=1}^{\infty} \alpha_k = \infty$ 且 $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$，并且 $\{x_k\}$ 是独立同分布的，那么 w_k 将几乎必然收敛到 $\mathbb{E}[X]$。值得一提的是，该收敛性质不依赖于 X 的分布假设。

6.1 罗宾斯-门罗算法

假设我们想要求解如下方程:

$$g(w) = 0,$$

其中 $w \in \mathbb{R}$ 是未知变量，$g : \mathbb{R} \to \mathbb{R}$ 是一个函数。如果 g 表达式是已知的，那么有很多数值算法可以用来求解 $g(w) = 0$。然而，我们面临的问题是函数 g 的表达式是未知的我们只知道该函数的输入和输入。具体来说，当输入 w 时，我们可以观测其输出值，而且该输出值可能还包含观测噪声:

$$\overline{g}(w,\eta) = g(w) + \eta,$$

其中 $\eta \in \mathbb{R}$ 是噪声，而且可能不是高斯分布。总而言之，该函数是一个黑盒系统。

求解 $g(w) = 0$ 的 RM 算法是

$$w_{k+1} = w_k - \alpha_k \overline{g}(w_k, \eta_k), \quad k = 1, 2, 3, \cdots$$

其中 w_k 是第 k 次方程解的估计值，$\overline{g}(w,\eta)$ 是输入 w_k 后输出的观测值，α_k 是一个正系数。

6.1.1 收敛性质

定理 6.1 (罗宾斯-门罗定理)**.** *在 RM 算法:*$w_{k+1} = w_k - \alpha_k(w_k - x_k)$ *中，如果下面三个条件全部成立:*

1. *存在两个常数* $c_1, c_2 > 0$，*使得* $0 < c_1 \le \nabla_w g(w) \le c_2$ *对于所有的* w *都成立;*
2. $\sum_{k=1}^{\infty} a_k = \infty$ *且* $\sum_{k=1}^{\infty} a_k^2 < \infty$;
3. $\mathbb{E}[\eta_k \mid \mathcal{H}_k] = 0$ *且* $\mathbb{E}[\eta_k^2 \mid \mathcal{H}_k] < \infty$，*其中* $\mathcal{H}_k = \{w_k, w_{k+1}, \cdots\}$.

那么 w_k *几乎必然收敛到* $g(w^*) = 0$ *的根* w^*。

下面我们讨论如何理解该定理中的三个条件:

1. 在条件 1 中，$0 < c_1 \le \nabla_w g(w)$ 要求 $g(w)$ 是一个单调递增的函数。这个确保条件 $g(w) = 0$ 的根存在且唯一。

   如果 $g(w)$ 是一个目标函数 $J(w)$ 的梯度，即 $g(w) := \nabla_w J(w)$，那么 $c_1 \le \nabla_w g(w)$ 等价于 $c_1 \le \nabla_w^2 J(w)$，这意味着 $J(w)$ 是凸的这是在优化问题中经常采用的假设，也说明该条件是比较宽松的。

   不等式 $\nabla_w g(w) \le c_2$ 表示 $g(w)$ 的梯度不能趋于无穷。否则有可能发散。

2. 条件 2 是关于 α_k 的，我们经常在强化学习算法中看到类似的条件。其中 $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$ 意味着 $\lim_{k\to\infty} \alpha_k = 0$。条件 $\sum_{k=1}^{\infty} \alpha_k = \infty$ 意味着 α_k 不能太快的收敛到 0。

3. 条件 3 是关于噪声 η_k 的，它并不要求观测误差 η_k 是高斯分布的。该条件成立的一个重要特例是 $\{\eta_k\}$ 是一个独立同分布的随机序列，因此满足 $\mathbb{E}[\eta_k] = 0$ 且 $\mathbb{E}[\eta_k^2] < \infty$。此时因为 η_k 与 $\mathcal{H}_k$ 是独立的，所以 $\mathbb{E}[\eta_k \mid \mathcal{H}_k] = \mathbb{E}[\eta_k] = 0$ 并且 $\mathbb{E}[\eta_k^2 \mid \mathcal{H}_k] = \mathbb{E}[\eta_k^2] < \infty$。

下面更加细致的讨论上述定理中关于 α_k 的条件，它对 RM 算法的收敛性是必要的。

第一，条件 $\sum_{k=1}^{\infty} \alpha_k^2 \le \infty$ 要求 $\lim_{k\to\infty} \alpha_k = 0$。这个条件为什么重要呢？由于

$$\lim_{k\to\infty} (w_{k+1} - w_k) = \lim_{k\to\infty} (-\alpha_k \overline{g}(w_k, \eta_k)) = 0,$$

它保证了 w_{k+1} 和 w_k 之间非常接近 (Cauchy 序列)。

第二，条件 $\sum_{k=1}^{\infty}\alpha_k=\infty$ 表明 α_k 不能太快收敛到 0。为什么这个条件这么重要呢？我们相加得到

$$w_1-w_\infty=\sum_{k=1}^{\infty}\alpha_k\overline{g}(w_k,\eta_k).$$

我们用反证法。假设 $\sum_{k=1}^{\infty}\alpha_k<\infty$，进而 $|\sum_{k=1}^{\infty}\alpha_k\overline{g}(w_k,\eta_k)|$ 也有上界 b，于是

$$|w_1-w_\infty|=\left|\sum_{k=1}^{\infty}\alpha_k\overline{g}(w_k,\eta_k)\right|\leq b$$

此时，如果我们选取的初始值 w_1 距离方程的最优解 w^* 很远，以至于 $|w_1-w^*|$，那么上述不等式告诉我们 $w_\infty=w^*$ 是不可能的，所以在这种情况下 RM 算法无法找到 w^*。因此，为了确保选取任意初始值时 RM 算法都能收敛，条件 $\sum_{k=1}^{\infty}\alpha_k=\infty$ 是必须的。

最经典的满足 $\mathbb{E}[\eta_k\mid\mathcal{H}_k]=0$ 且 $\mathbb{E}[\eta_k^2\mid\mathcal{H}_k]<\infty$ 的序列是

$$\alpha_k=\frac{1}{k}$$

因为

$$\sum_{k=1}^{\infty}\frac{1}{k}=\infty,\quad\sum_{k=1}^{\infty}\frac{1}{k^2}=\frac{\pi^2}{6}<\infty$$

第七章　时序差分方法

7.1　状态值估计：最基础的时序差分算法

本章节将介绍最基础的 TD 算法，它可以估计一个给定策略的状态值。

7.1.1　算法描述

给定一个策略 π，我们的目标是估计所有的 $s \in \mathcal{S}$ 的状态值 v_π。假设我们有一些由 π 生成的经验样本 $(s_0, r_1, s_1, \cdots, s_t, r_{t+1}, s_{t+1}, \cdots)$，其中 $t = 0, 1, 2, \cdots$ 表示采样时刻。下面的 TD 算法可以使用这些样本来估计状态值：

$$
\begin{aligned}
v_{t+1}(s_t) &= v_t(s_t) - \alpha_t(s_t)[v_t(s_t) - (r_t + \gamma v_t(s_{t+1}))], \\
v_{t+1}(s) &= v_t(s), \quad \text{当 } s \neq s_t
\end{aligned}
$$

其中 $v_t(s_t)$ 是在 t 时刻 $v_\pi(s_t)$ 的估计，$\alpha_t(s_t)$ 是 t 时刻对于状态 s_t 的学习率。

在时刻 t，只有当时正在被访问的状态 s_t 的估计值会被更新 (如 (A) 式)；而所有其它未被访问的状态的估计值保持不变 (如 (B) 式)。

为了理解 TD 算法为什么具有这样的形式，我们首先回顾一下状态值的定义：

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s], \quad s \in \mathcal{S}.$$

定理　7.1 (贝尔曼期望方程)**.** 上式可以重写为贝尔曼期望方程：

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s], \quad s \in \mathcal{S}$$

上式可以重写为贝尔曼期望方程：

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s], \quad s \in \mathcal{S}$$

证明. 已知

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

对第二部分 $E_\pi[G_{t+1} \mid S_t = s]$ 进行展开。根据全期望公式可得

$$E_\pi[G_{t+1} \mid S_t = s] = \sum_{a \in \mathcal{A}} \pi(a \mid s) \mathbb{E}_\pi[G_{t+1} \mid S_t = s,\, A_t = a]$$

给定 $S_t = s$ 和 $A_t = a$，下一状态 S_{t+1} 分布为 $p(s^{'} \mid s, a)$:

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s, A_t = a] = \sum_{s^{'}} p(s^{'} \mid s, a) \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s^{'}]$$

但根据 $v_\pi(s^{'})$ 的定义:

$$\mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s^{'}] = v_\pi(s^{'})$$

因此

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s, A_t = a] = \sum_{s^{'}} p(s^{'} \mid s, a) v_\pi(s^{'})$$

于是

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s] = \sum_a \pi(a \mid s) \sum_{s^{'}} p(s^{'} \mid s, a) v_\pi(s^{'}) = \mathbb{E}_\pi[v_\pi(S_{t+1}) \mid S_t = s]$$

为了证明上述等式的后半段，我们可以考虑:

$$\mathbb{E}[g(X) \mid Y = y] = \sum_x g(x) Pr(X = x \mid Y = y)$$

在这个问题中:

$$g = v_\pi,\, X = S_{t+1},\, Y = S_t,\, Pr(S_{t+1} = s^{'} \mid S_t = s) = \sum_a \pi(a \mid s) p(s^{'} \mid s, a)$$

所以

$$E_\pi[v_\pi(S_{t+1}) \mid S_t = s] = \sum_{s^{'}} v_\pi(s^{'}) \sum_a \pi(a \mid s) p(s^{'} \mid s, a)$$

□

推导时序差分算法

对于状态 s_t，定义函数:

$$g(v_\pi(s_t)) := v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s_t]$$

这样，贝尔曼方程可以写成:

$$g(v_\pi(s_t)) = 0.$$

我们的目的是求解上述方程来得到 $v_\pi(s_t)$。因为我们可以获取 r_{t+1} 和 s_{t+1}，而它们是 R_{t+1} 和 S_{t+1} 的样本，所以对 $g(v_\pi(s_t))$ 含有噪声的观测是:

$$\begin{aligned}\overline{g}(v_\pi(s_t)) = &\underbrace{(v_\pi(s_t) - \mathbb{E}[R_{t+1}\gamma + v_\pi(S_{t+1}) \mid S_t = s_t])}_{g(v_\pi(s_t))} + \\ &+ \underbrace{(\mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s_t] - [r_{t+1} + \gamma v_\pi(s_{t+1})])}_{\eta}\end{aligned}$$

此时用来求解 $g(v_\pi(s_t)) = 0$ 的 RM 算法是

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(v_t(s_t) - [r_{t+1} + \gamma v_\pi(s_t)]).$$

其中 $v_t(s_t)$ 是在时刻 t 对 $v_\pi(s_t)$ 的估计，而 $\alpha_t(s_t)$ 是学习率。

7.1.2 性质分析

第一，我们先介绍 TD 算法中每一项的含义。

$$\underbrace{v_{t+1}(s_t)}_{\text{新的估计值}} = \underbrace{v_t(s_t)}_{\text{当前估计值}} - \alpha_t(s_t)\overbrace{[v_t(s_t) - \underbrace{(r_{t+1} + \gamma v_t(s_{t+1}))}_{\text{TD 目标}}]}^{\text{TD 误差}},$$

其中

$$r_{t+1} + \gamma v_t(s_{t+1}) := \overline{v}_t$$

被称为 TD 目标，而

$$v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1})) = v_t(s_t) - \overline{v}_t := \delta_t$$

被称为 TD 误差。

Q1: 为什么 $\overline{v}_t$ 被称为 TD 目标?

这是因为该算法在数学上就是让 $v(s_t)$ 的值更接近 $\overline{v}_t$，即 $\overline{v}_t$ 是 $v(s_t)$ 的目标。为理解这一点，我们在迭代公式两边同时减去 $\overline{v}_t$:

$$\begin{aligned}v_{t+1}(s_t) - \overline{v}_t &= [v_t(s_t) - \overline{v}_t] - \alpha_t(s_t)[v_t(s_t) - \overline{v}_t] \\ &= [1 - \alpha_t(s_t)][v_t(s_t) - \overline{v}_t]\end{aligned}$$

两边同时取绝对值后:

$$\begin{aligned}|v_{t+1}(s_t) - \overline{v}_t| &= |[1 - \alpha_t(s_t)]| \cdot |[v_t(s_t) - \overline{v}_t]| \\ &\leq |v_t(s_t) - \overline{v}_t|\end{aligned}$$

这个不等式很清晰地说明了新的值 $v_{t+1}(s_t)$ 比旧的值 $v_t(s_t)$ 更接近 $\overline{v}_t$。

Q2: 如何理解 TD 误差?

TD 误差被称为 TD(时序差分) 的原因是 $\delta_t = v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))$ 反映了 t 和 $t+1$ 时刻之间的差异。TD 误差被称为“误差”的原因是它不仅仅反映了两个时刻之间的误差，更重要的是反应过了估计值 v_t 与真实状态值 v_π 之间的误差。如果估计值是准确的，那么 TD 误差在期望意义上应该等于 0。为了理解这一点，当 $v_t = v_\pi$ 时，TD 误差的期望值为 (δ_t 是 S_t 的函数，而 S_t 是随机变量)：

$$\begin{aligned}\mathbb{E}[\delta_t \mid S_t = s] &= \mathbb{E}[v_\pi(S_t) - (R_{t+1} + \gamma v_\pi(S_{t+1})) \mid S_t = s_t] \\ &= v_\pi(s_t) - \mathbb{E}[(R_{t+1} + \gamma v_\pi(S_{t+1})) \mid S_t = s_t] \\ &= 0 \quad (\text{贝尔曼期望方程})\end{aligned}$$

从另一角度来说，TD 误差可以被理解为新息 (innovation)，即代表从经验样本 (s_t, r_{t+1}, s_{t+1}) 中得到的新的信息，这个新的信息可以用来纠正当前估计值，从而使其更准确。

第二，TD 算法只能用来估计某一给定策略的状态值，而不能直接用于寻找最优策略。

第三，TD 算法和 MC 算法都是无模型算法，他们的区别在于：

表 7.1　TD 方法和 MC 方法的对比。

TD 方法	MC 方法
增量式：它可以在得到一个经验样本后立即更新估计值。	*非增量式*：它必须等到一个回合（episode）结束之后，才能用所有经验样本来更新估计值，这是因为它需要计算从某一状态到回合最后的折扣回报。
持续任务：由于 TD 算法是增量式的，因此它可以处理回合制（episodic）和持续性（continuing）的任务。	*回合制任务*：由于 MC 算法是非增量式的，因此它只能处理回合制任务，这些任务会在有限步后结束。
自举：TD 算法依赖于自举（bootstrapping），因为状态值/动作值的更新依赖于其先前估计值。因此，TD 算法需要初始值。	*非自举*：MC 算法不是自举的，因为它可以直接估计状态值/动作值，而无需初始值。
低估计方差：TD 算法的估计方差较低，这是因为它涉及的随机变量较少。例如，要估计动作值 $q_\pi(s_t, a_t)$，Sarsa 只需要三个随机变量 R_{t+1}、S_{t+1}、A_{t+1} 的样本。	*高估计方差*：MC 算法的估计方差较高，这是因为它涉及许多随机变量。例如，要估计动作值 $q_\pi(s_t, a_t)$，MC 算法需要 $R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$ 的样本。假设每个回合的步数为 L，并且每个状态的动作数等于 $\lvert\mathcal{A}\rvert$。那么，一个随机性的软策略可能有 $\lvert\mathcal{A}\rvert^L$ 种可能的轨迹。如果我们只用少数几个回合来估计，那么估计方差较高也就不足为奇了。

图 7.1: TD 算法和 MC 算法比较

7.1.3 TD 算法的收敛性*

该算法本质是一种特殊的 RM 算法，所以收敛性条件和 RM 算法一致。

定理 7.2 (TD 算法的收敛性)**.** *给定一个策略 π，对于 TD 算法，如果对所有的 $s \in \mathcal{S}$ 都有 $\sum_t \alpha_t(s) = \infty$ 和 $\sum_t \alpha_t^2(s) < \infty$，则 $v_t(s)$ 随着 $t \to \infty$ 几乎必然收敛到 $v_\pi(s)$。*

7.2 动作值估计：Sarsa

给定一个策略，我们的目标是估计其动作值。如果有一些由 π 生成的经验样本：$(s_0, a_0, r_1, s_1, a_1, \cdots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \cdots)$，那么可以使用下面的 Sarsa 算法来估计动作值：

$$
\begin{aligned}
&q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))],\\
&q_{t+1}(s_t, a_t) = q_t(s_t, a_t), \quad \text{当 } (s, a) \neq (s_t, a_t),
\end{aligned}
$$

其中 $q_t(s_t, a_t)$ 是 $q_\pi(s_t, a_t)$ 的估计值，$\alpha_t(s_t, a_t)$ 是学习率。

Sarsa 是一个用于求解如下所示的贝尔曼方程的随机近似算法：

$$
q_\pi(s, a) = \mathbb{E}[R + \gamma q_\pi(S', A') \mid s, a], \quad \forall (s, a).
$$

定理 7.3 (Sarsa 的收敛性)**.** *给定一个策略，对于 Sarsa 算法，如果 $\sum_t \alpha_t(s, a) = \infty$ 且 $\sum_t \alpha_t^2(s, a) < \infty$ 对于所有的 (s, a) 都成立，那么 $q_t(s, a)$ 随着 $t \to \infty$ 会几乎必然收敛到 $q_\pi(s, a)$。*

7.2.1 学习最优策略

Sarsa 算法只能估计一个给定策略的动作值。要想得到最优策略，我们需要将其与“策略改进步骤”相结合，结合之后的算法通常也被称为 Sarsa。其伪代码如下

可以看到，每次迭代有两个步骤：第一步是值更新；第二步是策略更新，即新的策略要选取最大价值的动作。值得注意的是，在值被更新之后，s_t 的策略会被立即更新，而不是在更新策略之前充分的评估当前策略，这也是广义策略迭代的思想。此外，在策略更新之后，该策略立即被用来生成下一个经验样本。这里的策略是 ϵ-Greedy 的，因此具有一定的探索性。

算法 7.1：用 Sarsa 学习最优策略

初始化： 对于所有 (s,a) 和所有 t，选取 $\alpha_t(s,a)=\alpha>0$。$\epsilon\in(0,1)$。所有 (s,a) 的初始值 $q_0(s,a)$。从 q_0 导出的初始 ϵ-Greedy 策略 π_0。

目标： 学习最优策略从而使智能体能从给定状态 s_0 出发到达目标状态。

对于每个回合

　在 s_0，根据 $\pi_0(s_0)$，得到 a_0

　在时刻 t，如果 s_t 不是目标状态

　　收集经验样本 $(s_t,a_t,r_{t+1},s_{t+1},a_{t+1})$：在 s_t，执行 a_t，通过与环境交互生成 r_{t+1},s_{t+1}，再根据 $\pi_t(s_{t+1})$ 生成 a_{t+1}

　　更新 (s_t,a_t) 的值：

$$q_{t+1}(s_t,a_t)=q_t(s_t,a_t)-\alpha_t(s_t,a_t)\Big[q_t(s_t,a_t)-(r_{t+1}+\gamma q_t(s_{t+1},a_{t+1}))\Big]$$

　　更新 s_t 的策略：

$$\pi_{t+1}(a|s_t)=1-\tfrac{\epsilon}{|\mathcal{A}(s_t)|}(|\mathcal{A}(s_t)|-1)，\text{如果 } a=\arg\max_a q_{t+1}(s_t,a)$$

$$\pi_{t+1}(a|s_t)=\tfrac{\epsilon}{|\mathcal{A}(s_t)|}，\text{如果 } a\neq\arg\max_a q_{t+1}(s_t,a)$$

　$s_t\leftarrow s_{t+1}$, $a_t\leftarrow a_{t+1}$

图 7.2: Sarsa 学习最优策略

7.3　n-step Sarsa*

7.4　最优动作值估计：Q-learning

本章将介绍 Q-learning 算法，这是经典的强化学习算法之一。前面介绍的 Sarsa 只能估计给定策略的动作值，必须结合策略改进步骤才能得到最优策略。相比之下，Q-learning 可以直接估计最优动作值进而找到最优策略。

7.4.1　算法描述

Q-learning 算法如下所示：

$$q_{t+1}(s_t,a_t)=q_t(s_t,a_t)-\alpha_t(s_t,a_t)\left[q_t(s_t,a_t)-(r_{t+1}+\gamma\max_{a\in\mathcal{A}}q_t(s_{t+1},a))\right],$$

$$q_{t+1}(s,a)=q_t(s,a),\quad 当\ (s,a)\neq(s_t,a_t),$$

Q-learning 的表达之与 Sarsa 非常相似，它们的区别在于 TD 目标：Q-learning 的 TD 目标是 $t_{t+1}+\gamma\max_a q_t(s_{t+1},a)$，而 Sarsa 的 TD 目标是

$r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$。因此当前的状态-动作是 (s_t, a_t)，Sarsa 算法的更新需要样本 $(r_{t+1}, s_{t+1}, a_{t+1})$，而 Q-learning 只需要 (r_{t+1}, s_{t+1})。

在数学上 Q-learning 是求解如下贝尔曼最优方程的随机近似算法：

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \mid S_t = s, A_t = a\right].$$

接下来我们介绍两个重要概念：Off-policy(异策略) 和 On-policy(同策略)。Q-learning 是 Off-policy 的，而前面介绍的算法如 Sarsa 都是 on-policy 的。

任何一个强化学习算法都会涉及两种策略：一种是行为策略，另一种是目标策略。行为策略用于生成经验样本，而目标策略不断更新，从而收敛到最优策略。当行为策略与目标策略相同时，该算法被称为 on-policy 的；当他们不同时，该算法被称为 off-policy 的。

off-policy 算法的优势在于它可以使用由其他策略生成的经验样本来学习最优策略。一个常见的情况时使用探索性较强的行为策略生成的经验数据。例如，如果我们想要估计所有动作值，则必须生成多次访问每个状态-动作的轨迹，此时可以用 ϵ-Greedy 策略来生成轨迹。尽管 Sarsa 也使用 ϵ-Greedy 策略来保持一定的探索能力，但是为了保证最优性，其 ϵ 的值通常很小，因此探索能力有限。相较之下，如果我们能使用一个具有较强探索能力的策略来生成经验数据，然后使用 off-policy 算法来学习最优策略，效率将显著提高。

最后，理清 online/offline 和 on-policy/off-policy 之间的关系。在线学习是指智能体在与环境交互的同时用生成的数据来更新值和策略。离线学习是指智能体不与环境交互，而是使用预先收集的数据来更新值和策略。如果算法是 on-policy 的，那么它可以实现在线学习，但不能离线学习，因为它无法使用预先收集的其他策略生成的数据。如果算法是 off-policy 的，那么它既可以在线学习，也可以离线学习。

7.4.2　算法实现

由于 Q-learning 是 off-policy 的，所以它在编程实现时有两种模式。

第一，on-policy 模式，即行为策略和目标策略相同，两者都使用 ϵ-Greedy 策略。第二，off-policy 模式，行为策略选择 ϵ-Greedy 策略，目标策略选择贪婪策略。

算法 7.2: Q-learning（On-policy 模式）

初始化： 对所有 (s,a) 和所有 t，$\alpha_t(s,a)=\alpha>0$。$\epsilon\in(0,1)$。所有 (s,a) 的初始值 $q_0(s,a)$。从 q_0 导出的初始 ϵ-Greedy 策略 π_0。

目标： 学习最优策略从而使智能体能从给定状态 s_0 出发到达目标状态。

对于每个回合

在 t 时刻，如果 s_t 不是目标状态

收集经验样本 (a_t, r_{t+1}, s_{t+1})：在 s_t，根据 $\pi_t(s_t)$ 产生 a_t，通过与环境互动生成 r_{t+1}, s_{t+1}。

更新 (s_t,a_t) 的值：

$q_{t+1}(s_t,a_t)=q_t(s_t,a_t)-\alpha_t(s_t,a_t)\Big[q_t(s_t,a_t)-(r_{t+1}+\gamma\max_a q_t(s_{t+1},a))\Big]$

更新 s_t 的策略：

$\pi_{t+1}(a|s_t)=1-\frac{\epsilon}{|\mathcal{A}(s_t)|}(|\mathcal{A}(s_t)|-1)$，如果 $a=\mathrm{argmax}_a q_{t+1}(s_t,a)$

$\pi_{t+1}(a|s_t)=\frac{\epsilon}{|\mathcal{A}(s_t)|}$，如果 $a\neq\mathrm{argmax}_a q_{t+1}(s_t,a)$

图 7.3: Q-learning(On-policy)

算法 7.3: Q-learning（Off-policy 模式）

初始化： 所有 (s,a) 的初始值 $q_0(s,a)$。所有 (s,a) 的行为策略 $\pi_b(a|s)$。对所有 (s,a) 和所有 t，$\alpha_t(s,a)=\alpha>0$。

目标： 使用 π_b 生成的经验数据，学习所有状态的最优策略 π_T。

对 π_b 生成的每个回合 $\{s_0,a_0,r_1,s_1,a_1,r_2,\dots\}$

对回合中的每一步 $t=0,1,2,\dots$

更新 (s_t,a_t) 的值：

$q_{t+1}(s_t,a_t)=q_t(s_t,a_t)-\alpha_t(s_t,a_t)\Big[q_t(s_t,a_t)-(r_{t+1}+\gamma\max_a q_t(s_{t+1},a))\Big]$

更新 s_t 的目标策略：

$\pi_{T,t+1}(a|s_t)=1$，如果 $a=\mathrm{argmax}_a q_{t+1}(s_t,a)$

$\pi_{T,t+1}(a|s_t)=0$，如果 $a\neq\mathrm{argmax}_a q_{t+1}(s_t,a)$

图 7.4: Q-learning(Off-policy)

第八章　值函数方法

本章仍然讲解时序差分方法，不过我们将用函数来表示状态值或者动作值。其基本思想就是函数拟合，找到一个函数 $v_\pi(s,w)$ 使得其在样本点 $\{(s_i,\overline{v}_\pi(s_i))\}_{i=1}^n$ 上的误差最小。这样可以用参数 w 储存整个值函数。并且某一个状态值更新会牵动整个值函数进行优化。

8.1　基于值函数的时序差分算法：状态值估计

下面介绍如何将值函数与时序差分方法相结合，实现对一个给定策略的状态值的估计。

8.1.1　目标函数

令 $v_\pi(s)$ 和 $\overline{v}(s,w)$ 分别表示状态 $s\in\mathcal{S}$ 的真实状态值和估计状态值。我们的任务时找到一个最优的参数 w，从而使得 $\overline{v}(s,w)$ 能够最好的近似每一个 s 的 $v_\pi(s)$。具体来说，目标函数是

$$J(w)=\mathbb{E}[(v_\pi(S)-\overline{v}(S,w))^2],$$

其中 $S\in\mathcal{S}$ 是随机变量。由于 S 是一个随机变量，那么它的概率分布是什么呢? 这是本书第一次将状态描述成随机变量并且需要刻画其概率分布，这也是使用值函数时需要解决的重要问题。

本章节重点介绍平稳分布。平稳分布描述了马尔可夫决策过程的长期行为。更具体地说，当智能体执行一个给定策略足够长的时间后，智能体位于任意一个状态的概率都可以由这个平稳分布来描述。

具体来说，设 $\{d_\pi(s)\}_{s\in\mathcal{S}}$ 为在策略 π 下的平稳分布。此时目标函数可以重写为

$$J(w)=\sum_{s\in\mathcal{S}}d_\pi(s)(v_\pi(s)-\overline{v}(s,w))^2.$$

这是所有状态的估计误差的加权平均，那些有更高概率被访问到的状态被赋予了更大的权重。具体平稳分布的存在性以及计算，见随机过程课本。

8.1.2 优化算法

为了最小化目标函数 $J(w)$，我们可以使用梯度下降算法：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

其中梯度是

$$\begin{aligned}\nabla_w J(w) &= \nabla_w \mathbb{E}[(v_\pi(S) - \overline{v}(S,w))^2] \\ &= \mathbb{E}[\nabla_w (v_\pi(S) - \overline{v}(S,w))^2] \\ &= 2\mathbb{E}[(v_\pi(S) - \overline{v}(S,w))(-\nabla_w \overline{v}(S,w))] \\ &= -2\mathbb{E}[(v_\pi(S) - \overline{v}(S,w))\nabla_w \overline{v}(S,w)]\end{aligned}$$

将上面梯度代入梯度下降算法可得

$$w_{k+1} = w_k + 2\alpha_k \mathbb{E}[(v_\pi(S) - \overline{v}(S,w_k))\nabla_w \overline{v}(S,w_k)],$$

但是上述算法不能直接使用，因为它需要真实期望值，而真实期望值在实际中很难得到。此时，我们可以使用随机梯度代替真实梯度，这是随机梯度下降算法的思想。那么上式将变为

$$w_{t+1} = w_k + \alpha_t (v_\pi(s_t) - \overline{v}(s_t,w_k))\nabla_w \overline{v}(s_t,w_t),$$

其中 s_t 是 t 时刻得到的 S 的一个样本。

但是上述算法仍然是无法直接使用的，因为它需要真实的状态价值 v_π，这是未知的也正是我们需要估计的。此时，我们可以用一个近似值替代 $v_\pi(s_t)$，具体来说有下面两种方法：

1. 蒙特卡罗方法：如果我们有一个从 s_t 开始的回合数据，设 g_t 为从 s_t 开始的折扣回报，那么 g_t 可以作为 $v_\pi(s_t)$ 的近似值。此时，算法变为

   $$w_{t+1} = w_t + \alpha_t (g_t - \overline{v}(s_t,w_t))\nabla_w \overline{v}(s_t,w_t).$$

   这是基于值函数的蒙特卡罗算法。

2. 时序差分方法：根据时序差分的思想，我们可以用 TD 目标 $r_{t+1} + \gamma \overline{v}(s_{t+1},w_t)$ 来代替真实值函数 $v_\pi(s_t)$。此时，算法变为

   $$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \overline{v}(s_{t+1},w_t) - \overline{v}(s_t,w_t)]\nabla_w \overline{v}(s_t,w_t).$$

这是基于值函数的 TD 算法，伪代码如下

算法 8.1: 基于值函数的 TD 算法（用于状态值估计）

初始化: 参数可微的值函数 $\hat{v}(s,w)$。初始参数 w_0。

目标: 估计一个给定策略 π 的状态值。

对于由 π 生成的每个回合 $\{(s_t, r_{t+1}, s_{t+1})\}_t$

　对于每个样本 (s_t, r_{t+1}, s_{t+1})

　对于一般值函数：$w_{t+1} = w_t + \alpha_t[r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)]\nabla_w \hat{v}(s_t, w_t)$

　对于线性值函数：$w_{t+1} = w_t + \alpha_t\big[r_{t+1} + \gamma\phi^{\mathrm{T}}(s_{t+1})w_t - \phi^{\mathrm{T}}(s_t)w_t\big]\phi(s_t)$

图 8.1: 基于值函数的 TD 算法

8.1.3　选择值函数

为了应用上述 TD 算法，我们需要选择合适的值函数 $\overline{v}(s,w)$。目前最常见的是使用人工神经网络：神经网络的输入是状态 s，输出是 $\overline{v}(s,w)$，网络参数是 w。下面重点介绍历史上早期使用较广泛的线性函数，其优点是具有较强的理论可解释性，其劣势是具有较弱的近似能力，并且在实际中往往难以选取合适的特征向量。

具体来说，一个线性函数具有如下形式：

$$\overline{v}(s,w) = \phi^\top(s)w,$$

其中 $\phi(s) \in \mathbb{R}^m$ 是状态 s 的特征向量。$\phi(s)$ 和 w 的维数都等于 m，而 m 通常远小于状态的个数。线性函数的梯度非常简单：

$$\nabla_w \overline{v}(s,w) = \phi(s).$$

于是

$$w_{t+1} = w_t + \alpha_t[r_{t+1} + \gamma\phi^\top(s_{t+1})w_t - \phi^\top(s_t)w_t]\phi(s_t).$$

这是基于线性值函数的 TD 算法。

8.1.4　理论分析

下面我们对线性值函数 TD 算法进行分析。

收敛性分析

为了研究上述算法的收敛性，我们首先考虑如下算法：

$$w_{t+1} = w_t + \alpha_t\mathbb{E}[(r_{t+1} + \gamma\phi^\top(s_{t+1})w_t - \phi^\top(s_t)w_t)\phi(s_t)], \tag{A}$$

其中的期望是针对三个随机变量 s_t, s_{t+1}, r_{t+1}。上述算法是确定性的，因为所有随机变量在计算期望之后都消失了。

这个确定性算法的收敛性更容易分析，并且该确定性算法的收敛性能够推导出基于值函数的 TD 算法的收敛性。为了简化上述表达式，假设 s_t 服从平稳分布 d_π，定义

$$\Phi = \begin{bmatrix} \vdots \\ \phi^\top(s) \\ \vdots \end{bmatrix} \in \mathbb{R}^{n\times m}, \quad D = \begin{bmatrix} \ddots & & \\ & d_\pi(s) & \\ & & \ddots \end{bmatrix} \in \mathbb{R}^{n\times n},$$

其中矩阵 Φ 的每一行对应一个状态的特征向量，对角矩阵 D 的对角元素是平稳分布向量中的元素。

引理 8.1. 中的期望可以重写为

$$\mathbb{E}[(r_{t+1} + \gamma\phi^\top(s_{t+1})w_t - \phi^\top(s_t)w_t)\phi(s_t)] = b - Aw_t$$

其中

$$\begin{aligned} A &:= \Phi^\top D(I - \gamma P_\pi)\Phi \in \mathbb{R}^{m\times m}, \\ b &:= \Phi^\top D r_\pi \in \mathbb{R}^m. \end{aligned}$$

这里 P_π, r_π 是贝尔曼方程 $v_\pi = r_\pi + \gamma P_\pi v_\pi$ 中的两个量，而 I 是具有适合维度的单位矩阵。

证明. 设 s_t 服从平稳分布 d_π，使用全期望公式可得：

$$\begin{aligned} &\mathbb{E}[r_{t+1}\phi(s_t) + \phi(s_t)(\gamma\phi^\top(s_{t+1}) - \phi^\top(s_t))w_t] \\ =&\sum_{s\in\mathcal{S}} d_\pi(s)\mathbb{E}[r_{t+1}\phi(s_t) + \phi(s_t)(\gamma\phi^\top(s_{t+1}) - \phi^\top(s_t))w_t \mid s_t = s] \\ =&\sum_{s\in\mathcal{S}} d_\pi(s)\mathbb{E}[r_{t+1}\phi(s_t) \mid s_t = s] + \sum_{s\in\mathcal{S}} d_\pi(s)\mathbb{E}[\phi(s_t)(\gamma\phi^\top(s_{t+1}) - \phi^\top(s_t))w_t \mid s_t = s] \end{aligned}$$

第一，考虑上式第一项。由于

$$\mathbb{E}[r_{t+1}\phi(s_t) \mid s_t = s] = \phi(s)\mathbb{E}[r_{t+1} \mid s_t = s] = \phi(s)r_\pi(s),$$

其中 $r_\pi(s) = \sum_a \pi(a \mid s) r p(r \mid s, a)$，因此第一项可以重写为

$$\sum_{s\in\mathcal{S}} d_\pi(s)\mathbb{E}[r_{t+1}\phi(s_t) \mid s_t = s] = \sum_{s\in\mathcal{S}} d_\pi(s)\phi(s)r_\pi(s) = \Phi^\top D r_\pi,$$

第二，考虑第二项，由于

$$\begin{aligned} &\mathbb{E}[\phi(s_t)(\gamma\phi^\top(s_{t+1}) - \phi^\top(s_t))w_t \mid s_t = s] \\ =&\gamma\phi(s)\mathbb{E}[\phi^\top(s_{t+1}) \mid s_t = s]w_t - \phi(s)\phi^\top(s)w_t \\ =&\gamma\phi(s)\sum_{s'\in\mathcal{S}} p(s' \mid s)\phi^\top(s')w_t - \phi(s)\phi^\top(s)w_t \end{aligned}$$

因此第二项变为

$$
\begin{aligned}
&\sum_{s\in\mathcal{S}} d_\pi(s)\mathbb{E}[\phi(s_t)(\gamma\phi^\top(s_{t+1})-\phi^\top(s_t))w_t \mid s_t=s] \\
=&\sum_{s\in\mathcal{S}} d_\pi(s)\left[\gamma\phi(s)\sum_{s'\in\mathcal{S}} p(s' \mid s)\phi^\top(s')w_t-\phi(s)\phi^\top(s)w_t\right] \\
=&\sum_{s\in\mathcal{S}} d_\pi(s)\phi(s)\left[\gamma\sum_{s'\in\mathcal{S}} p(s' \mid s)\phi^\top(s')-\phi(s)\right]w_t \\
=&\Phi^\top D(-\Phi+\gamma P_\pi\Phi)w_t \\
=&-\Phi^\top D(I-\gamma P_\pi)\Phi w_t
\end{aligned}
$$

两式结合;

$$
\begin{aligned}
\mathbb{E}[r_{t+1}\phi(s_t)+\phi(s_t)(\gamma\phi^\top(s_{t+1})-\phi^\top(s_t))w_t] &= \Phi^\top Dr_\pi-\Phi^\top D(I-\gamma P_\pi)\Phi w_t \\
&= b-Aw_t
\end{aligned}
$$

其中

$$
\begin{aligned}
A &:= \Phi^\top D(I-\gamma P_\pi)\Phi \in \mathbb{R}^{m\times m}, \\
b &:= \Phi^\top Dr_\pi \in \mathbb{R}^m.
\end{aligned}
$$

□

根据上述引理的表达式，确定性算法可以重写为

$$w_{t+1}=w_t+\alpha_t(b-Aw_t).$$

这是一个确定性迭代算法，其收敛性分析如下所示。

1. 第一，假设 w_t 会收敛到一个常值 w^*，并且满足

   $$w^*=w_*+\alpha_\infty(b-Aw^*)\longrightarrow w^*=A^{-1}b$$

   关于这个收敛值，给出下面几个说明:1. A 可逆且正定；2. $w^*=A^{-1}b$ 是最小化投影贝尔曼误差的最优解。

2. 下面证明算法中的 w_t 会随着 $t\to\infty$ 收敛到 $w^*=A^{-1}b$。

TD-linear 算法优化的是投影贝尔曼误差

上一节我们证明了 TD-Linear 算法收敛于 $w^*=A^{-1}b$。下面我们将证明 TD-Linear 算法实际上是在最小化投影贝尔曼误差，而 w^* 就是最优解。为此我们梳理三个目标函数。

1. 第一个目标函数是

$$J(w) = \mathbb{E}[(v_\pi(S) - \overline{v}(S, w))^2].$$

本章最开始就是使用这个目标函数来介绍值函数方法的思路的，该目标函数也可以等价地写成一个矩阵-向量形式：

$$J_E(w) = ||\overline{v}(w) - v_\pi||_D^2,$$

其中 v_π 是真实状态值向量，而 $\overline{v}(w)$ 是估计的值向量，这两个向量的每一个元素都对应一个状态。这里 $||\cdot||_D^2$ 是加权范数：

$$||x||_D^2 := x^\top Dx = ||D^{1/2}x||_2^2.$$

该目标函数是我们能想到的最简单的目标函数之一。然而，这个目标函数涉及未知的真实状态值，所以直接优化它是无法得到可行算法的。因此我们必须考虑其他目标函数。

2. 第二个目标函数是贝尔曼误差。具体来说，由于 v_π 满足贝尔曼方程 $v_\pi = r_\pi + \gamma P_\pi v_\pi$，因此估计值 $\overline{v}(w)$ 也应尽可能满足此方程。贝尔曼误差定义：

$$J_{BE}(w) = ||\overline{v}(w) - (r_\pi + \gamma P_\pi \overline{v}(w))||_D^2 := ||\overline{v}(w) - T(\overline{v}(w))||_D^2.$$

上式中 $T_\pi(\cdot)$ 是贝尔曼算子：对任意 $x \in \mathbb{R}^n$ 有

$$T_\pi(x) := r_\pi + \gamma P_\pi x$$

最小化贝尔曼误差就是一个标准的最小二乘问题。但是由于函数的近似能力有限，该目标函数可能无法被最小化到 0。

3. 第三个目标函数是投影贝尔曼误差，其定义为

$$J_{PBE}(w) = ||\overline{v}(w) - MT(\overline{v}(w))||_D^2.$$

其中 $M \in \mathbb{R}^{n\times n}$ 是一个正交投影矩阵，它在几何上可以将任意向量投影到函数能够近似的值空间上。在线性情况下，矩阵 M 的表达式为：

$$M = \Phi(\Phi^\top D\Phi)^{-1}\Phi D \in \mathbb{R}^{n\times n}$$

该矩阵可以将任意向量投影到 Φ 的列空间，由于 $MT_\pi(\overline{v}(w))$ 在 Φ 的列空间中，于是一定可以找到列空间中的一个向量 $\overline{v}(w)$ 使得 $J_{PBE}(w)$ 最小化为 0.

定理 8.2. $J_{PBE}(w)$ 的最优解是

$$w^* = A^{-1}b = \arg\min_w J_{PBE}(w).$$

证明. 由于 $J_{PBE}(w) = 0$ 等价于 $\overline{v}(w) - MT_\pi(\overline{v}(w)) = 0$，因此我们只需要求解

$$\overline{v}(w) = MT_\pi(\overline{v}(w)).$$

在线性情况下，将 $\overline{v}(w) = \Phi w$ 和上述 M 的表达式代入可得

$$\Phi w == \Phi(\Phi^\top D\Phi)^{-1}\Phi D(r_\pi + \gamma P_\pi \Phi w).$$

由于 Φ 列满秩，因此

$$\begin{aligned}
&w = (\Phi^\top D\Phi)^{-1}\Phi D(r_\pi + \gamma P_\pi \Phi w)\\
&\Leftrightarrow \Phi D(r_\pi + \gamma P_\pi \Phi w) = (\Phi^\top D\Phi)w\\
&\Leftrightarrow \Phi^\top D r_\pi + \gamma \Phi^\top D P_\pi \Phi w = (\Phi^\top D\Phi)w\\
&\Leftrightarrow \Phi^\top D r_\pi = \Phi^\top D(I - \gamma P_\pi)\Phi w\\
&\Leftrightarrow w = (\Phi^\top D(I - \gamma P_\pi)\Phi)^{-1}\Phi^\top D r_\pi = A^{-1}b
\end{aligned}$$

□

由于 TD-Linear 算法旨在最小化 J_{PBE} 而并非 J_E，我们自然会问其之间误差为多少？下面这个定理估计了其上界大小。

定理 8.3. 在线性的情况下，最小化 J_{PBE} 的最优估计值是 $\overline{v}(w^*) = \Phi w$，其与真实的状态值 v_π 的误差满足如下不等式：

$$||\Phi w^* - v_\pi||_D \le \frac{1}{1-\gamma}\min_v ||\overline{v}(w) - v_\pi||_D = \frac{1}{1-\gamma}\min_w \sqrt{J_E(w)}$$

8.1.5 基于值函数的 Sarsa

8.1.6 基于值函数的 Q-Learning

8.2 深度 Q-Learning

其基本思想是用深度神经网络作为值函数进行训练，具体操作过程有些许改进。

第九章　策略梯度方法

当用函数表示策略时，我们的任务是最大化一个标量目标函数 $J(\theta)$，其中 θ 表示策略函数的参数。最简单的优化方法是梯度上升：

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\theta_t),$$

其中 $\nabla_\theta J$ 是 J 相对 θ 的梯度，$\alpha > 0$ 是步长。

9.1　目标函数：定义最优策略

在策略梯度方法中，用于定义最优策略的目标函数有如下两种。

目标函数 1: 平均状态值

第一个常见的目标函数是平均状态值，其定义为

$$\overline{v}_\pi = \sum_{s \in \mathcal{S}} d(s) v_\pi(s),$$

其中 $d(s)$ 是状态 s 的权重，它满足对任何 $s \in \mathcal{S}$ 都有 $d(s) \geq 0$ 且 $\sum_{s \in \mathcal{S}} d(s) = 1$，显然它是一个关于 s 的概率分布。那么 m 该目标函数可以重写为

$$\overline{v}_\pi = \mathbb{E}_{S \sim d}[v_\pi(S)].$$

顾名思义，$\overline{v}_\pi$ 是所有状态值的加权平均。我们的任务是找到一个最优策略 (θ) 来最大化目标函数 $\overline{v}_\pi$。

观察到该目标函数中包含概率分布 $d(s)$，有两种常见的情况。

1. 第一，d 与策略 π 无关，此时该目标函数对策略参数求梯度不需要考虑 d。在此情况下，我们特地用 d_0 来替代 d, 用 $\overline{v}_\pi^0$ 来替代 $\overline{v}_\pi$，以表明该概率分布与策略无关。

   例如，如果我们认为所有状态的重要性相同，那么可以选择 $d_0(s) = 1/|\mathcal{S}|$。如果我们只对某个特定状态 s_0 感兴趣 (例如智能体始终从 s_0 出

发)，那么可以设计

$$d_0(s_0) = 1, \quad d_0(s \neq s_0) = 0.$$

2. 第二，d 与策略 π 有关。此时常见的选择是将 d 设为 d_π，即在 π 下的平稳分布。d_π 的一个基本性质是 $d_\pi^\top P_\pi = d_\pi^\top$，其中 P_π 是状态转移概率矩阵。

下面介绍 $\overline{v}_\pi$ 的两个等价表达式。

1. 等价表达式 1: 假设智能体根据给定策略 $\pi(\theta)$ 收集了一个奖励序列 $\{R_{t+1}\}_{t=0}^\infty$，则

$$\begin{aligned} J(\theta) &= \mathbb{E}\left[\sum_{t=0}^\infty \gamma^t R_{t+1}\right] \\ &= \sum_{s\in\mathcal{S}} d(s)\mathbb{E}\left[\sum_{t=0}^\infty \gamma^t R_{t+1} \mid S_0 = s\right] \\ &= \sum_{s\in\mathcal{S}} d(s) v_\pi(s) \\ &= \overline{v}_\pi \end{aligned}$$

2. 等价表达式 2: 目标函数 $\overline{v}_\pi$ 也可以重写为两个向量的内积。令

$$\begin{aligned} v_\pi &= [\cdots, v_\pi(s), \cdots]^\top \in \mathbb{R}^{|\mathcal{S}|}, \\ d &= [\cdots, d(s), \cdots]^\top \in \mathbb{R}^{|\mathcal{S}|}. \end{aligned}$$

那么有

$$\overline{v}_\pi = d^\top v_\pi$$

这个表达式在分析其梯度时十分有用。

目标函数 2：平均奖励

第二个常见的目标函数就是平均奖励，它的定义是：

$$\overline{r}_\pi := \sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}_{S\sim d_\pi}[r_\pi(S)],$$

其中 d_π 是平稳分布，另外：

$$r_\pi(s) := \sum_{a\in\mathcal{A}} \pi(a \mid s, \theta) r(s,a) = \mathbb{E}_{A\sim\pi(s,\theta)}[r(s,A) \mid s]$$

是从状态 s 出发的单布即时奖励的期望值。这里 $r(s,a) := \mathbb{E}[R \mid s,a] = \sum_r rp(r \mid s,a)$。下面介绍两个 $\overline{r}_\pi$ 的两个等价表达式。特别是第一个表达式，大家在文献中经常看见。

1. 等价表达式 1：假设智能体根据给定策略 $\pi(\theta)$ 收集到了一个奖励序列 $\{R_{t+1}\}_{t=0}^{\infty}$，大家可能经常在文献中看见如下目标函数：

$$J(\theta) = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right].$$

   虽然这个目标函数乍一看很复杂，但是它实际就是平均奖励 $\overline{r}_\pi$，这是因为

$$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right] = \sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s) = \overline{r}_\pi$$

2. 向量形式:

$$\overline{r}_\pi = d_\pi^\top r_\pi$$

引理 9.1 (Cesaro 均值)**.** *如果 $\{a_k\}_{k=1}^{\infty}$ 是一个收敛序列并且极限 $\lim_{k\to\infty} a_k$ 存在，那么 $\{1/n\sum_{k=1}^{n} a_k\}_{k=1}^{\infty}$ 也是一个收敛序列，并且*

$$\lim_{n\to\infty} \frac{1}{n}\sum_{k=1}^{n} a_k = \lim_{k\to\infty} a_k.$$

定理 9.2. *等价表达式 1 和原始等价，即：*

$$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}\right] = \sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s) = \overline{r}_\pi$$

总结

9.2 目标函数的梯度

定理 9.3 (策略梯度定理)**.** *$J(\theta)$ 的梯度是*

$$\nabla_\theta J(\theta) = \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a),$$

其中 η 是状态的概率分布，π_θ 是 π 关于 θ 的梯度。此外上式还有如下等价形式：

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta, A\sim\pi(S,\theta)}\left[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)\right],$$

其中 ln 是自然对数。

上述两个形式的等价性是因为:

$$\nabla_\theta \ln \pi(a \mid s,\theta) = \frac{\nabla_\theta \pi(a \mid s,\theta)}{\pi(a \mid s,\theta)} \Leftrightarrow \nabla_\theta \pi(a \mid s,\theta) = \pi(a \mid s,\theta)\nabla_\theta \ln \pi(a \mid s,\theta)$$

第一步，证明以下方程对任何起始状态 $s_0 \in \mathcal{S}$ 都是成立的：

$$\bar{r}_\pi = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}|S_0 = s_0\right]. \tag{9.6}$$

为此，首先注意到

$$\begin{aligned}\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1} R_{t+1}|S_0 = s_0\right] &= \lim_{n\to\infty} \frac{1}{n}\sum_{t=0}^{n-1} \mathbb{E}\left[R_{t+1}|S_0 = s_0\right] \\ &= \lim_{t\to\infty} \mathbb{E}\left[R_{t+1}|S_0 = s_0\right], \end{aligned} \tag{9.7}$$

其中最后一个等号是根据 Cesaro 均值的性质（也称为 Cesaro 求和）。具体来说，如果 $\{a_k\}_{k=1}^{\infty}$ 是一个收敛序列并且极限 $\lim_{k\to\infty} a_k$ 存在，那么 $\{1/n\sum_{k=1}^{n} a_k\}_{n=1}^{\infty}$ 也是一个收敛序列，并且有 $\lim_{n\to\infty} 1/n\sum_{k=1}^{n} a_k = \lim_{k\to\infty} a_k$。

下面分析式(9.7)中的 $\mathbb{E}\left[R_{t+1}|S_0 = s_0\right]$。根据总期望定律可得

$$\begin{aligned}\mathbb{E}\left[R_{t+1}|S_0 = s_0\right] &= \sum_{s\in\mathcal{S}} \mathbb{E}\left[R_{t+1}|S_t = s, S_0 = s_0\right] p^{(t)}(s|s_0) \\ &= \sum_{s\in\mathcal{S}} \mathbb{E}\left[R_{t+1}|S_t = s\right] p^{(t)}(s|s_0) \\ &= \sum_{s\in\mathcal{S}} r_\pi(s) p^{(t)}(s|s_0),\end{aligned}$$

其中 $p^{(t)}(s|s_0)$ 表示从 s_0 开始后恰好使用 t 步转移到 s 的概率。上式中的第二个等号是由于马尔可夫性质：下一时刻获得的奖励只依赖于当前状态而与之前的状态无关。根据平稳分布的定义可得

$$\lim_{t\to\infty} p^{(t)}(s|s_0) = d_\pi(s).$$

于是

$$\begin{aligned}\nabla_\theta J(\theta) &= \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a) \\ &= \mathbb{E}_{S\sim\eta}\left[\sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid S,\theta) q_\pi(S,a)\right] \\ &= \mathbb{E}_{S\sim\eta}\left[\sum_{a\in\mathcal{A}} \pi(a \mid S,\theta) \nabla_\theta \ln \pi(a \mid S,\theta) q_\pi(S,a)\right] \\ &= \mathbb{E}_{S\sim\eta,\, A\sim\pi(S,\theta)}\left[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)\right]\end{aligned}$$

其中的 $\pi(a \mid s,\theta)$ 因为在 ln 函数里面，所以要求为正数，并且不能为 1，因此这个策略是随机并且具有探索性的（即 $0 < \pi(a \mid s,\theta) < 1$），这可以通过

上式表明不论从哪个状态出发，最终转移到 s 的概率都是 $d_\pi(s)$。因此，我们有

$$\lim_{t\to\infty}\mathbb{E}\left[R_{t+1}|S_0=s_0\right]=\lim_{t\to\infty}\sum_{s\in\mathcal{S}}r_\pi(s)p^{(t)}(s|s_0)=\sum_{s\in\mathcal{S}}r_\pi(s)d_\pi(s)=\bar{r}_\pi.$$

将上式代入式(9.7)可得式(9.6)。

第二步，考虑任意的状态分布向量 d。根据总期望定律有

$$\begin{aligned}\lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1}R_{t+1}\right]&=\lim_{n\to\infty}\frac{1}{n}\sum_{s\in\mathcal{S}}d(s)\mathbb{E}\left[\sum_{t=0}^{n-1}R_{t+1}|S_0=s\right]\\&=\sum_{s\in\mathcal{S}}d(s)\lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1}R_{t+1}|S_0=s\right].\end{aligned}$$

将式(9.6)代入上述方程可得

$$\lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1}R_{t+1}\right]=\sum_{s\in\mathcal{S}}d(s)\bar{r}_\pi=\bar{r}_\pi.$$

上式中第二个等号是因为 $\sum_{s\in\mathcal{S}}d(s)=1$。证明完毕。

是用 Softmax 函数实现:

$$\pi(a\mid s,\theta)=\frac{e^{h(s,a,\theta)}}{\sum_{a'\in\mathcal{A}}e^{h(s,a',\theta)}},\quad a\in\mathcal{A},$$

其中 $h(a,s,\theta)$ 是一个特征函数，表示状态 s 选择动作 a 的有限度。

9.2.1　推导策略梯度：有折扣的情况

首先我们考虑有折扣的情况，即 $\gamma\in(0,1)$。首先我们知道

$$v_\pi(s)=\sum_{a\in\mathcal{A}}\pi(a\mid s,\theta)q_\pi(s,a)$$

首先我们证明 $\overline{v}_\pi(\theta)$ 与 $\overline{r}_\pi(\theta)$ 是等价的目标函数。

引理　9.4 ($\overline{v}_\pi(\theta)$ 与 $\overline{r}_\pi(\theta)$ 等价)**.** *在有折扣的情况下，即 $\gamma\in(0,1)$ 时，有*

$$\overline{r}_\pi=(1-\gamma)\overline{v}_\pi$$

因此，$\overline{r}_\pi$ 和 $\overline{v}_\pi$ 能被同时最大化。

证明. 注意到 $\overline{v}_\pi=d_\pi^\top v_\pi$ 并且 $\overline{r}_\pi=d_\pi^\top r_\pi$，其中 v_π 和 r_π 满足贝尔曼方程 $v_\pi=r_\pi+\gamma P_\pi v_\pi$。两边同时乘以 $d_{\pi]}{}^\top$ 可得

$$\overline{v}_\pi=d_\pi^\top v_\pi=d_\pi^\top r_\pi+\gamma d_\pi^\top P_\pi v_\pi=\overline{r}_\pi+\gamma d_\pi^\top v_\pi=\overline{r}_\pi+\gamma\overline{v}_\pi$$

到目前为止，我们介绍了两种目标函数：$\bar{v}_\pi$ 和 $\bar{r}_\pi$。每种目标函数都有几种不同但等价的表达式，见表9.2。另外，我们有时用 $\bar{v}_\pi$ 特指状态分布是稳定分布 d_π 的情况，而用 $\bar{v}_\pi^0$ 特指分布 d_0 与 π 无关的情况。下面是对这些目标函数的一些补充说明。

表 9.2　$\bar{v}_\pi$ 和 $\bar{r}_\pi$ 的不同但等价的表达式。

目标函数	表达式1	表达式2	表达式3
$\bar{v}_\pi$	$\sum_{s\in\mathcal{S}} d(s)v_\pi(s)$	$\mathbb{E}_{S\sim d}[v_\pi(S)]$	$\lim_{n\to\infty}\mathbb{E}\left[\sum_{t=0}^{n}\gamma^t R_{t+1}\right]$
$\bar{r}_\pi$	$\sum_{s\in\mathcal{S}} d_\pi(s)r_\pi(s)$	$\mathbb{E}_{S\sim d_\pi}[r_\pi(S)]$	$\lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[\sum_{t=0}^{n-1}R_{t+1}\right]$

◇ 第一，所有这些目标函数都是 π 的函数。由于 π 是由 θ 参数化的，因此这些目标函数是 θ 的函数。不同的 θ 值会得到不同的目标函数值，我们的任务是寻找最优的 θ 来最大化这些目标函数，这就是策略梯度方法的基本思想。

◇ 第二，这两个目标函数 $\bar{v}_\pi$ 和 $\bar{r}_\pi$ 在 $\gamma<1$ 的情况下是等价的，这是因为

$$\bar{r}_\pi=(1-\gamma)\bar{v}_\pi.$$

上式表明这两个目标函数可以同时被最大化，因此我们不需要纠结究竟选用哪个

图 9.1: 策略目标函数

于是可得

$$\overline{r}_\pi=(1-\gamma)\overline{v}_\pi$$

□

第二，下面的引理给出了任意一个状态值对策略的梯度。

引理　9.5 (状态值的梯度)**.** 在有折扣的情况下，即 $\gamma\in(0,1)$ 时，对任意 $s\in\mathcal{S}$ 都有

$$\nabla_\theta\pi_\pi(s)=\sum_{s^{'}\in\mathcal{S}}Pr_\pi(s^{'}\mid s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a\mid s^{'},\theta)q_\pi(s^{'},a),$$

其中

$$Pr_\pi(s^{'}\mid s):=\sum_{k=0}^{\infty}\gamma^k[P_\pi^k]_{ss'}=[(I-\gamma P_\pi)^{-1}]_{ss'}$$

是在策略 π 下从状态 s 转移到状态 $s^{'}$ 的折扣总概率。

证明. 首先对于任意的 $s \in \mathcal{S}$ 有

$$\begin{aligned}\nabla_\theta v_\pi(s) &= \nabla_\theta \left[\sum_{a\in\mathcal{A}} \pi(a \mid s,\theta) q_\pi(s,a)\right] \\ &= \sum_{a\in\mathcal{A}} [\nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a) + \pi(a \mid s,\theta)\nabla_\theta q_\pi(s,a)],\end{aligned}$$

其中

$$\begin{aligned}\nabla_\theta q_\pi(s,a) &= \nabla_\theta \left[\sum_{r\in\mathcal{R}} rp(r \mid s,a) + \gamma \sum_{s'\in\mathcal{S}} p(s^{'} \mid s,a) v_\pi(s^{'})\right] \\ &= \gamma \sum_{s'\in\mathcal{S}} p(s^{'} \mid s,a)\nabla_\theta v_\pi(s^{'}).\end{aligned}$$

代入上式可得

$$\begin{aligned}\nabla_\theta v_\pi &= \sum_{a\in\mathcal{A}} \left[\nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a) + \pi(a \mid s,\theta)\gamma \sum_{s'\in\mathcal{S}} p(s^{'} \mid s,a)\nabla_\theta v_\pi(s^{'})\right] \\ &= \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a) + \gamma \sum_{a\in\mathcal{A}} \pi(a \mid s,\theta) \sum_{s'\in\mathcal{S}} p(s^{'} \mid s,a)\nabla_\theta v_\pi(s^{'})\end{aligned}$$

利用矩阵-向量形式表示。首先设

$$u(s) := \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a)$$

其次有

$$\sum_{a\in\mathcal{A}} \pi(a \mid s,\theta) \sum_{s'\in\mathcal{S}} p(s^{'} \mid s,a)\nabla_\theta v_\pi(s^{'}) = \sum_{s'\in\mathcal{S}} p(s^{'} \mid s)\nabla_\theta v_\pi(s^{'}) = \sum_{s'\in\mathcal{S}} [P_\pi]_{ss'}\nabla_\theta v_\pi(s^{'}),$$

因此上述式子的矩阵-向量形式为

$$\nabla_\theta v_\pi = u + \gamma(P_\pi \otimes I_m)\nabla_\theta v_\pi.$$

上述线性方程的解为

$$\nabla_\theta v_\pi = (I_{mn} - \gamma P_\pi \otimes I_m)^{-1} u = [(I_n - \gamma P_\pi)^{-1} \otimes I_m]u$$

下面给出针对状态 s 的展开形式

$$\nabla_\theta v_\pi(s) = \sum_{s'\in\mathcal{S}} [(I_n - \gamma P_\pi)^{-1}]_{ss'} \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a)$$

对于上述式子中的 $[(I_n - \gamma P_\pi)^{-1}]_{ss'}$ 我们可以展开为

$$[(I_n - \gamma P_\pi)^{-1}]_{ss'} = [I]_{ss'} + \gamma[P_\pi]_{ss'} + \gamma^2[P_\pi]^2_{ss'} + \cdots = \sum_{k=0}^{\infty} \gamma^k [P_\pi^k]_{ss'}$$

□

基于上述引理，下面推导 $\overline{v}_\pi^0$ 的梯度，这里的上标 0 表示该目标函数中的状态概率分布与策略 π 无关。

定理 9.6 (有折扣的 $\overline{v}_\pi^0$ 的梯度)**.** *在有折扣的情况下，即当 $\gamma \in (0,1)$ 时，$\overline{v}_\pi^0 = d_0^\top v_\pi$ 的梯度是*

$$\nabla_\theta \overline{v}_\pi^0 = \mathbb{E}_{S\sim\rho_\pi,\, A\sim\pi(S,\theta)}[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)],$$

其中

$$\rho_\pi(s) = \sum_{s'\in\mathcal{S}} d_0(s') Pr_\pi(s \mid s')$$

其中 $Pr_\pi(s \mid s') := \sum_{k=0}^{\infty} \gamma^k [P_\pi^k]_{ss'} = [(I-\gamma P_\pi)^{-1}]_{ss'}$。

证明. 对 $\overline{v}_\pi^0$ 求导得

$$\begin{aligned}
\nabla_\theta \overline{v}_\pi^0 &= \sum_{s\in\mathcal{S}} d_0(s) \nabla_\theta v_\pi(s) \\
&= \sum_{s\in\mathcal{S}} d_0(s) \sum_{s'\in\mathcal{S}} Pr_\pi(s' \mid s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s',\theta) q_\pi(s',a) \\
&= \sum_{s'\in\mathcal{S}} \left(\sum_{s\in\mathcal{S}} d_0(s) Pr_\pi(s' \mid s) \right) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s',\theta) q_\pi(s',a) \\
&= \sum_{s'\in\mathcal{S}} \rho_\pi(s') \sum_{a\in\mathcal{A}} \pi(a \mid s',\theta) \nabla_\theta \ln(\pi(a \mid s',\theta)) q_\pi(s',a) \\
&= \mathbb{E}_{S\sim\rho_\pi,\, A\sim\pi(S,\theta)}[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)].
\end{aligned}$$

其中

$$\rho_\pi(s) = \sum_{s'\in\mathcal{S}} d_0(s') Pr_\pi(s \mid s').$$

□

定理 9.7 (有折扣下 $\overline{v}_\pi$ 和 $\overline{r}_\pi$ 的梯度)**.** *在有折扣的情况下，即当 $\gamma \in (0,1)$ 时，$\overline{v}_\pi$ 和 $\overline{r}_\pi$ 的梯度为*

$$\begin{aligned}
\nabla_\theta \overline{r}_\pi = (1-\gamma)\nabla_\theta \overline{v}_\pi &\approx \sum_{s\in\mathcal{S}} d_\pi(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a) \\
&= \mathbb{E}_{S\sim d_\pi,\, A\sim\pi(S,\theta)}[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)].
\end{aligned}$$

当 γ 接近 1 时，上面的近似更加准确。

9.2.2 推导策略梯度：无折扣的情况

定理 9.8 (无折扣下 $\overline{v}_\pi$ 和 $\overline{r}_\pi$ 的梯度)**.** 在无折扣的情况下，$\overline{v}_\pi$ 和 $\overline{r}_\pi$ 的梯度为

$$\begin{aligned}\nabla_\theta \overline{r}_\pi = (1-\gamma)\nabla_\theta \overline{v}_\pi &= \sum_{s\in\mathcal{S}} d_\pi(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a) \\ &= \mathbb{E}_{S\sim d_\pi,\, A\sim\pi(S,\theta)}[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)].\end{aligned}$$

由于 v_π 不是唯一的，所以我们常用 $\overline{r}_\pi$。

9.3 蒙特卡罗策略梯度

有了上一节给出的目标函数的梯度，我们就可以使用如下的梯度上升算法来最大化目标函数以获得最佳策略：

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha_t \nabla_\theta J(\theta) \\ &= \theta_t + \alpha_t \mathbb{E}[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)],\end{aligned}$$

其中 $\alpha > 0$ 是学习率。由于算法中的真实梯度含有期望，我们可以用随机梯度替代真实梯度，从而得到如下算法：

$$\theta_{t+1} = \theta_t + \alpha_t \nabla_\theta \pi(a_t \mid s_t,\theta_t) q_t(a_t,s_t), \tag{!}$$

其中 $q_t(s_t,a_t)$ 是对 $q_\pi(s_t,a_t)$ 在 t 时刻的估计。

算法 (!) 非常重要，因为许多其他策略梯度算法都可以通过推广该算法得到。从表达式可以看出，策略参数 θ_t 的更新依赖于对动作值的估计 $q_t(s_t,a_t)$。到目前为止本书介绍了两种估计值的方法，一种是蒙特卡罗模拟，另一种是时序差分方法。根据 $q_t(a_t,s_t)$ 估计方法的不同，算法 (!) 分为蒙特卡罗策略梯度和 *Actor-Critic* 方法。

下面仔细地分析算法 (!)。由于

$$\nabla_\theta \ln \pi(a_t \mid s_t,\theta_t) = \frac{\nabla_\theta \pi(a_t \mid s_t,\theta_t)}{\pi(a_t \mid s_t,\theta_t)},$$

算法 (!) 可重写为

$$\theta_{t+1} = \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t,a_t)}{\pi(a_t \mid s_t,\theta_t)}\right)}_{\beta_t} \nabla_\theta \pi(a_t \mid s_t,\theta_t)$$

上式可以简写为

$$\theta_{t+1} = \theta_t + \alpha \beta_t \nabla_\theta \pi(a_t \mid s_t,\theta_t).$$

从上面这个方程可以得到重要结论。当 $\theta_{t+1}-\theta_t$ 足够小时，根据一阶泰勒展开：

$$\begin{aligned}\pi(a_t \mid s_t,\theta_{t+1}) &\approx \pi(a_t \mid s_t,\theta_t)+(\nabla_\theta \pi(a_t \mid s_t,\theta_t))^\top(\theta_{t+1}-\theta_t)\\ &= \pi(a_t \mid s_t,\theta_t)+\alpha\beta_t||\nabla_\theta \pi(a_t \mid s_t,\theta_t)||_2^2.\end{aligned}$$

当 $\beta \geq 0$ 时，$\pi(a_t \mid s_t,\theta_{t+1}) \geq \pi(a_t \mid s_t,\theta_t)$，即在 s_t 选择 a_t 的概率会增大；当 $\beta_t < 0$ 时，$\pi(a_t \mid s_t,\theta_{t+1}) < \pi(a_t \mid s_t,\theta_t)$，即在 s_t 选择 a_t 的概率会下降。

计算随机梯度样本 A 服从分布 $\pi(S,\theta)$，样本 S 服从 η。

算法 9.1：蒙特卡罗策略梯度（REINFORCE）

初始化： 初始参数 θ；$\gamma \in (0,1)$；$\alpha > 0$。

目标： 学习一个最优策略从而最大化 $J(\theta)$。

对于每个回合

　根据 $\pi(\theta)$ 生成 $\{s_0,a_0,r_1,\ldots,s_{T-1},a_{T-1},r_T\}$。

　对于 $t=0,1,\ldots,T-1$：

　　价值更新：$q_t(s_t,a_t)=\sum_{k=t+1}^{\mathrm{T}}\gamma^{k-t-1}r_k$

　　策略更新：$\theta \leftarrow \theta+\alpha\nabla_\theta \ln \pi(a_t|s_t,\theta)q_t(s_t,a_t)$

图 9.2: 蒙特卡罗策略梯度

第十章　Actor—Critic 方法

10.1　最简单的 A-C 方法：QAC

就是将蒙特卡罗策略梯度中 $q_t(a_t, s_t)$ 的计算方式变为时序差分法。

10.2　优势 Act-Critic 方法

这个算法的核心思想是引入一个基准来减少估计的方差。

10.2.1　基准不变性

策略梯度有一个重要性质：它对额外的基准 *(baseline)* 是不变的，即

$$\mathbb{E}_{S\sim\eta,\,A\sim\pi}[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)] = \mathbb{E}_{S\sim\eta,\,A\sim\pi}[\nabla_\theta \ln \pi(A \mid S,\theta)(q_\pi(S,A) - b(S))]$$

其中 $b(S)$ 是基准函数，它是 S 的一个标量函数。上述式子说明了添加或去掉基准函数不会影响策略梯度。下面我们证明上述等式成立：

$$\begin{aligned}
\mathbb{E}_{S\sim\eta,\,A\sim\pi}[\nabla_\theta \ln \pi(A \mid S,\theta) b(S)] &= \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \pi(a \mid s,\theta) \nabla_\theta \ln \pi(a \mid s,\theta) b(s) \\
&= \sum_{s\in\mathcal{S}} \eta(s) b(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) \\
&= \sum_{s\in\mathcal{S}} \eta(s) b(s) \nabla_\theta \sum_{a\in\mathcal{A}} \pi(a \mid s,\theta) \\
&= \sum_{s\in\mathcal{S}} \eta(s) b(s) \nabla_\theta 1 = 0
\end{aligned}$$

上述式子说明加减基准函数不会影响期望，但是加减适当的基准函数后可以减小随机梯度对真实梯度近似的方差从而加速收敛。

定理　10.1. 能最小化随机梯度方差的基准函数为

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}[||\nabla_\theta \ln \pi(A \mid s,\theta)||^2 q_\pi(s,A)]}{\mathbb{E}_{A\sim\pi}[||\nabla_\theta \ln \pi(A \mid s,\theta)||^2]}, \quad s \in \mathcal{S}$$

证明. 令 $\overline{x} := \mathbb{E}[X]$。如果 X 是一个**向量**，那么其方差 $var(X)$ 是一个矩阵。通常可以选择其迹作为优化的标量目标函数。因为

$$\begin{aligned} tr(var[X]) &= tr\mathbb{E}[(X-\overline{x})(X-\overline{x})^\top] \\ &= tr\mathbb{E}[XX^\top - X\overline{x}^\top - \overline{x}X^\top + \overline{x}^\top\overline{x}] \\ &= \mathbb{E}[X^\top X - X^\top\overline{x} - \overline{x}X^\top + \overline{x}^\top\overline{x} \\ &= \mathbb{E}[X^\top X] - \overline{x}^\top\overline{x} \end{aligned}$$

如果 $\overline{x}$ 是不变的，那么最小化上式我们只需要最小化 $\mathbb{E}[X^\top X]$ 即可。于是我们需要处理的问题实际上是：

$$\mathbb{E}[X^\top X] = \mathbb{E}[(\nabla_\theta \ln\pi)^\top(\nabla_\theta \ln\pi)(q_\pi(S,A)-b(S))^2] = \mathbb{E}[||\nabla_\theta \ln\pi||^2(q_\pi(S,A)-b(S))^2]$$

目标函数最优的条件应该满足 $\nabla_b\mathbb{E}[X^\top X]=0$，即对任意的 $s\in\mathcal{S}$，$b(S)$ 应该满足：

$$\mathbb{E}_{A\sim\pi}[||\nabla_\theta \ln\pi||^2(q_\pi(s,a)-b(s))] = 0$$

不难求解上述方程进而得到最优基准函数：

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}[||\nabla_\theta \ln\pi(A\mid s,\theta)||^2 q_\pi(s,A)]}{\mathbb{E}_{A\sim\pi}[||\nabla_\theta \ln\pi(A\mid s,\theta)||^2]}, \quad s\in\mathcal{S}$$

□

尽管上述式子是最优的，但是它太复杂，在实际中无法使用。如果从上式中移除权重 $||\nabla_\theta \ln\pi(A\mid s,\theta)||^2$，就可以得到一个次优的基准函数：

$$b^{'}(s) = \mathbb{E}_{A\sim\pi}[q_\pi(s,A)] = v_\pi(s), \quad s\in\mathcal{S}.$$

值得注意的是，这个次优的基准函数就是状态值函数。

将次优基准函数应用到梯度上升算法中得到：

$$\begin{aligned} \theta_{t+1} &= \theta_t + \alpha\mathbb{E}[\nabla_\theta \ln\pi(A\mid S,\theta)[q_\pi(S,A)-v_\pi(S)]] \\ &= \theta_t + \alpha\mathbb{E}[\nabla_\theta \ln\pi(A\mid S,\theta)\delta_\pi(S,A)] \end{aligned}$$

其中

$$\delta_\pi(S,A) := q_\pi(S,A) - v_\pi(S)$$

被称为优势函数，它反映了一个动作相对于其他动作的优势。具体来说，由于状态值 $v_\pi(s)=\sum_{a\in\mathcal{A}}\pi(a\mid s)q_\pi(s,a)$ 是平均动作值，因此 $\delta_\pi(s,a)>0$ 意味着相应的动作值大于均值，具有一定的优势。

10.3 Off-policy Act-critic 方法

迄今为止，我们学习的策略梯度方法都是 *On-policy* 的，其原因可以从真实梯度的表达式中看出：

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta,\, A\sim\pi}[\nabla_\theta \ln \pi(A \mid S, \theta)(q_\pi(S, A) - v_\pi(S))].$$

为了利用蒙特卡罗方法使得随机梯度可以近似真是梯度 *(*带有期望的*)*，我们必须按照 $\pi(\theta)$ 来生成动作样本。

如果我们想要利用一些由其他策略生成的样本，我们需要使用重要性采样的技术。这个技术是通过使用根据某一概率分布得到的样本来估计另一概率分布的期望值的一种通用技术。

10.3.1 重要性采样

考虑一个随机变量 $X \in \mathcal{X}$。假设 $p_0(X)$ 是一个概率分布，我们的目标是估计 $\mathbb{E}_{X\sim p_0}[X]$。假设我们有一些独立同分布的样本 $\{x_i\}_{i=1}^n$，但是样本不是由 p_0 生成的，而是由另一个概率分布 p_1 生成的。这是我们需要用如下公式计算 $\mathbb{E}_{X\sim p_0}[X]$：

$$\mathbb{E}_{X\sim p_0}[X] = \sum_{x\in\mathcal{X}} p_0(x)x = \sum_{x\in\mathcal{X}} p_1(x) \underbrace{\frac{p_0(x)}{p_1(x)}x}_{f(x)} = \mathbb{E}_{X\sim p_1}[f(X)].$$

上式表明，估计 $\mathbb{E}_{X\sim p_0}[X]$ 被转换为估计 $\mathbb{E}_{X\sim p_1}[f(X)]$ 的问题。此时令*:*

$$\overline{f} = \frac{1}{n}\sum_{i=1}^n f(x_i)$$

于是

$$\mathbb{E}_{X\sim p_0}[X] = \mathbb{E}_{X\sim p_1}[f(X)] \approx \overline{f} = \frac{1}{n}\sum_{i=1}^n f(x_i) = \frac{1}{n}\sum_{i=1}^n \underbrace{\frac{p_0(x_i)}{p_1(x_i)}}_{\text{重要性权重}} x_i.$$

上式表明 $\mathbb{E}_{X\sim p_0}[X]$ 可以通过 x_i 的加权平均来近似，而这里的权重就是 $\frac{p_0(x_i)}{p_1(x_i)}$，它被称为重要性权重。当 $p_1 = p_0$ 时，重要性权重为 1，$\overline{f}$ 就变成了 $\overline{x}$。当 $p_0(x_i) \geq p_1(x_i)$ 时，这意味着 x_i 可以在 p_0 下更频繁地被采样到，而在 p_1 下较少地被采样到。

10.3.2 Off-policy 策略梯度定理

假设 β 是一个行为策略，我们的目标是使用由 β 生成的样本来得到一个目标策略 π，从而最大化下面的目标函数：

$$J(\theta) = \sum_{s \in \mathcal{S}} d_\beta(s) v_\pi(s) = \mathbb{E}_{S \sim d_\beta}[v_\pi(S)],$$

其中 d_β 是策略 β 下的平稳分布，v_π 是策略 π 下的状态值。次目标函数的梯度在下述定理中给出。

定理 10.2 (Off-policy 策略梯度定理)**.** 如果 $\gamma \in (0,1)$，那么 $J(\theta)$ 的 *off-policy* 梯度为

$$\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \rho,\, A \sim \beta}\left[\underbrace{\frac{\pi(A \mid S, \theta)}{\beta(A \mid S)}}_{\text{重要性权重}} \nabla_\theta \ln \pi(A \mid S, \theta) q_\pi(S, A)\right],$$

其中状态分布 ρ 为

$$\rho(s) := \sum_{s' \in \mathcal{S}} d_\beta(s^{'}) Pr_\pi(s \mid s^{'}), \quad s \in \mathcal{S}.$$

这里 $Pr_\pi(s \mid s^{'}) = \sum_{k=0}^{\infty} \gamma^k [P_\pi^k]_{ss'}$。

10.4 确定性 Act-Critic 方法

第十一章　现代强化学习算法

由于神经网络的引入，采用神经网络函数化值函数（*DQL*）和策略函数（*TRPO,PPO,GRPO,DPO* 等），可以极大地增强强化学习算法可处理问题的复杂度。针对策略函数，我们本质上是设计一个有效的优化算法去解决一个关于目标函数的最大值问题。对于一个用神经网络函数化的策略函数，最经典的处理方法是梯度下降。之前的策略梯度法中我们介绍了最基本的方法，随着时间的发展，更多基于策略梯度的优化方法被提出。

11.1　自然梯度下降

传统的梯度下降算法如下：

$$w_{t+1} = w_t - \alpha_t \nabla_w J(w)$$

但是这样子做有一个问题：参数 w 的改进不一定能够导致目标函数 $J(w)$ 的改进。导致这个问题的原因是：传统的梯度下降依赖于一个基本假设–参数 w 所在的参数空间是欧式空间。

11.1.1　欧氏空间中的梯度下降算法

对于目标函数 $J(w)$，参数 w 在一个欧式空间内，该欧式空间中的距离为

$$\mathrm{dist}(w, w+\delta w)^2 = ||\delta w||^2 = \sum_i (\delta w_i)^2$$

这个定义将所有参数 w_i 一视同仁，独立的衡量它们的变化。

最速下降方向：梯度下降的目标是在一个固定大小的欧式球（即满足约束 $||\delta w|| \leq \epsilon$）内，找到一个能使损失函数 $L(w+\delta w)$ 下降最快的方向 δw。通过一阶的泰勒展开：

$$L(w+\delta w) \approx L(w) + \nabla L(w)^\top \delta w$$

这个问题可以被描述为

$$\min_{\delta w} L(w) + \nabla L(w)^\top \delta w \quad s.t. \quad ||\delta w|| \leq \epsilon$$

这个优化问题的解就是

$$\delta w = -\epsilon \frac{\nabla L(w)}{||\nabla L(w)||}$$

将系数与 α 合并，就能得到我们熟悉的梯度下降更新公式：

$$w_{t+1} = w_t - \eta \nabla L(w_t).$$

11.1.2 黎曼空间中的梯度下降算法

同样，我们希望能在黎曼空间中找到一个度量，以衡量一个黎曼参数空间中两个元素之间的距离。针对强化学习，尤其是策略梯度下降，我们实际上关心的问题是参数按照某个度量进行更新后，目标函数是否协同更新。为了找到这样的度量，我们需要一个用来衡量更新前后目标函数（本质上是一个概率分布）变化的距离函数。

KL 散度定义了一个黎曼空间中良好的度量

在概率分布空间中，衡量两个分布 $p(x \mid w)$ 和 $p(x \mid w + \delta w)$ 之间的“距离”的一个自然选择是 KL 散度。

$$\begin{aligned}
D_{KL}(p(x \mid w) \,||\, p(x \mid w + \delta w)) &= \int p(x \mid w) \log \frac{p(x \mid w)}{p(x \mid w + \delta w)} \mathrm{d}x \\
&= \int p(x \mid w) \log p(x \mid w) \mathrm{d}x \\
&+ \int p(x \mid w) \log p(x \mid w + \delta w) \mathrm{d}x \\
&= \mathbb{E}_{p(x|w)}[\log p(x \mid w)] + \mathbb{E}_{p(x|w)}[\log p(x \mid w + \delta w)]
\end{aligned}$$

虽然 KL 散度不是一个严格的距离，但是在局部 δw 很小时，它可以近似为一个对称的二次型。

首先对 $\log p(x \mid w + \delta w)$ 在 w 处泰勒展开至二阶项：

$$\begin{aligned}
\log p(x \mid w + \delta w) \approx \log p(x \mid w) &+ (\nabla_{\delta w} \log p(x \mid w))^\top \delta w \\
&+ \frac{1}{2} (\delta w)^\top \nabla^2_{\delta w} \log p(x \mid w) \delta w + O(\delta w^3)
\end{aligned}$$

将上式代入 *KL* 散度:

$$
\begin{aligned}
D_{KL}(p(x \mid w) \mid\mid p(x \mid w+\delta w)) \approx & \int p(x \mid w)[\log p(x \mid w)-\log p(x \mid w) \\
& -(\nabla_{\delta w} \log p(x \mid w))^{\top} \delta w+\frac{1}{2}(\delta w)^{\top} \nabla_{\delta w}^{2} \log p(x \mid w) \delta w] \mathrm{d} x \\
= & -\int p(x \mid w)(\nabla_{\delta w} \log p(x \mid w))^{\top} \delta w \mathrm{d} x \\
& -\int p(x \mid w) \frac{1}{2}(\delta w)^{\top} \nabla_{\delta w}^{2} \log p(x \mid w) \delta w \mathrm{d} x
\end{aligned}
$$

对于第一项

$$
\begin{aligned}
& \int p(x \mid w)(\nabla_{\delta w} \log p(x \mid w))^{\top} \delta w \mathrm{d} x \\
= & \left[\int p(x \mid w) \nabla_{\delta w} \log p(x \mid w) \mathrm{d} x\right]^{\top} \delta w \\
= & \left[\int \nabla_{\delta w} p(x \mid w) \mathrm{d} x\right]^{\top} \delta w \\
= & \nabla_{\delta w}\left[\int p(x \mid w) \mathrm{d} x\right]^{\top} \delta w \\
= & (\nabla_{\delta w} 1) \delta w=0
\end{aligned}
$$

上式中的第一项为 0。于是 *KL* 散度在 w 处可以近似表示为

$$
\begin{aligned}
D_{KL}(p(x \mid w) \mid\mid p(x \mid w+\delta w)) \approx & -\int p(x \mid w) \frac{1}{2}(\delta w)^{\top} \nabla_{\delta w}^{2} \log p(x \mid w) \delta w \mathrm{d} x \\
= & -\frac{1}{2} \delta w^{\top}\left[\int p(x \mid w) \nabla_{\delta w}^{2} \log p(x \mid w) \mathrm{d} x=\mathbb{E}_{p(x \mid w)}\right] \delta w
\end{aligned}
$$

中间项

$$
\int p(x \mid w) \nabla_{\delta w}^{2} \log p(x \mid w) \mathrm{d} x=\mathbb{E}_{p(x \mid w)}[\nabla_{\delta w}^{2} \log p(x \mid w)]:=D
$$

D 一定是一个对称半正定矩阵，所以 *KL* 散度在 w 处的二次型构成了一个内积，说明当 δw 很小时，*KL* 散度确实是一个良好定义的距离。

KL 散度与 Fisher 信息矩阵之间的关系

如果选取 $s(w)=\nabla_{w} \log p(x \mid w)$ 作为参数 w 的得分函数，即我们要最大化 $p(x \mid w)$ 关于参数 w 的概率。可以得到得分函数的期望是 0*(之前证明过)*:

$$
\mathbb{E}_{p(x \mid w)}[s(w)]=\mathbb{E}_{p(x \mid w)}[\nabla_{w} \log p(x \mid w)]=0
$$

其次 $s(w)$ 的方差用来衡量估计的准确程度:

$$
\mathbb{E}_{p(x \mid w)}[(s(w)-0)(s(w)-0)^{\top}]
$$

这种准确程度的衡量，可以看作是信息量大小的衡量，因此我们把这个东西定义为信息矩阵，即 *Fisher* 矩阵：

$$F := \mathbb{E}_{p(x|w)}[\nabla \log p(x \mid w) \nabla \log p(x \mid w)^\top]$$

下面我们证明 *Fisher* 矩阵与 *KL* 散度的二阶近似项矩阵 D 满足如下关系：

$$F = -D$$

证明. 首先我们从得分函数期望为 0 可以得出:

$$\mathbb{E}_{p(x|w)}[\nabla \log p(x \mid w)] = \int p(x \mid w) \nabla \log p(x \mid w) \mathrm{d}x = 0 \qquad (*)$$

于是

$$\nabla_w \left[\int p(x \mid w) \nabla \log p(x \mid w) \mathrm{d}x \right] = 0$$

展开得到

$$\begin{aligned}
LHS &= \int \nabla [p(x \mid w) \nabla \log p(x \mid w)] \mathrm{d}x \\
&= \int \nabla p(x \mid w) \nabla \log p(x \mid w) \mathrm{d}x + \int p(x \mid w) \nabla^2 \log p(x \mid w) \mathrm{d}x \\
&= \int p(x \mid w) \nabla \log p(x \mid w) \nabla \log p(x \mid w)^\top + \int p(x \mid w) \nabla^2 \log p(x \mid w) \mathrm{d}x \\
&= \mathbb{E}_{p(x|w)}[\nabla \log p(x \mid w) \nabla \log p(x \mid w)^\top] + \mathbb{E}_{p(x|w)}[\nabla^2 \log p(x \mid w)] \\
&= F + D
\end{aligned}$$

根据 $(*)$ 可得

$$F + D = 0 \Rightarrow D = -F$$

□

于是，*KL* 散度与 *Fisher* 信息矩阵之间的关系为

$$D_{KL}(p(x \mid w) \mid\mid p(x \mid w + \delta w)) \approx \frac{1}{2} \delta w^\top F(w) \delta w + O(||\delta w||^3)$$

黎曼度量下的最速下降法与自然梯度

已知

$$D_{KL}(p(x \mid w) \mid\mid p(x \mid w + \delta w)) \approx \frac{1}{2} \delta w^\top F(w) \delta w + O(||\delta w||^3)$$

现在求解下降方向 δw 变为如下优化问题：

$$\delta w^* = \arg \min_{D_{KL} = c} L(w + \delta w)$$

利用拉格朗日乘子法求极值，并将 $L(w+\delta w)$ 一阶泰勒展开：

$$\delta w^* \approx \arg\min_{\delta w} L(w) + \nabla_w L(w)^\top \delta w + \frac{1}{2}\lambda(\delta w)^\top F(w)(\delta w) - \lambda c := f(\delta w, \lambda).$$

令目标函数的一阶导数为 0，我们可以得出：

$$\nabla_{\delta w} f(\delta w, \lambda) = \nabla_w L(w) + \lambda F(\delta w) = 0 \Rightarrow \delta w^* = -\frac{1}{\lambda} F^{-1} \nabla_w L(w)$$

其中 λ 是固定常数。利用线搜索确定步长，也就是将最优搜索方向 δw^* 带入目标函数，对 λ 求导使得目标函数最小：

$$\nabla_\lambda f(\delta w, \lambda) = \frac{1}{2}(\delta w)^\top F(w)(\delta w) - c = 0 \Rightarrow \frac{1}{2}(\delta w)^\top F(w)(\delta w) = c$$

将 $\delta w^* = -\frac{1}{\lambda} F^{-1} \nabla_w L(w)$ 带入上述表达式得到

$$c = \frac{1}{2\lambda^2} \nabla_w L(w)^\top F^{-1} \nabla_w L(w)$$

解得

$$\lambda = \sqrt{\frac{\nabla_w L(w)^\top F^{-1} \nabla_w L(w)}{2c}}$$

于是参数 w 的更新公式为

$$w_{t+1} = w_t + \sqrt{\frac{2c}{\nabla_w L(w_t)^\top F^{-1} \nabla_w L(w_t)}} F^{-1}(w_t) \nabla_w L(w_t)$$

11.2　信赖域策略梯度

之前讲的自然梯度下降存在三个问题：

1. 由于过程中涉及很多近似步骤，可能导致最终 *KL* 散度并不满足步长约束；
2. 确定梯度下降方向时需要计算 $F^{-1}(w)$，这一步计算复杂度很高 $O(n^3)$；
3. 我们并没有检查策略是否真的得到了优化。

第二个问题可以使用共轭梯度法求解矩阵的逆。针对第三个问题我们介绍从优势函数出发建立的 *TRPO* 算法。首先给出符号说明：考虑一个无限折扣的马尔可夫决策过程，由一数组 $(\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma)$ 决定，其中

1. $\mathcal{S}$ 是有限的状态空间，$\mathcal{A}$ 是有限的动作空间；
2. $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ 是一个转移概率分布；
3. $r : \mathcal{S} \to \mathbb{R}$ 是奖励函数；
4. $\rho_0 : \mathcal{S} \to \mathbb{R}$ 是是初始状态 s_0 的概率分布函数；
5. $\gamma \in (0, 1)$ 是折扣因子。

11.2.1 目标函数

设 $\pi : \mathcal{S} \times \mathcal{A} \to [0,1]$ 是随机策略，$\eta(\pi)$ 表示为期望折扣奖励函数：

$$\eta(\pi) = \mathbb{E}_{s_0, a_0, \cdots}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t)\right]$$

我们的目标是最大化上述期望折扣奖励函数。

11.2.2 优势函数

在之前讲解优势 *Act-Critic* 算法时我们提到一个性质——基准不变性，当基准函数选为次优基准函数 $v_\pi(s)$ 时，则 $A_\pi(s) = q_\pi(s,a) - v_\pi(s)$ 定义为优势函数，用来反应这个一个动作值相较于其他的平均动作值是否有明显优势。这里的优势函数有相同定义：

$$Q_\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}, a_{t+1}, \cdots}\left[\sum_{l=0}^{\infty} \gamma^l r(s_{t+l})\right]$$

$$V_\pi(s_t) = \mathbb{E}_{a_t, s_{t+1}, \cdots}\left[\sum_{l=0}^{\infty} \gamma^l r(s_{t+l})\right]$$

优势函数定义为：

$$A_\pi(s,a) = Q_\pi(s,a) - V_\pi(s)$$

其中 $a_t \sim \pi(a_t \mid s_t)$, $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$。下面我们介绍一个引理，它说明了两个策略之间的差值可以用优势函数来刻画。

引理 11.1. 给定两个策略 $\pi, \overline{\pi}$

$$\eta(\overline{\pi}) = \eta(\pi) + \mathbb{E}_{\tau \sim \overline{\pi}}\left[\sum_{t=0}^{\infty} \gamma^t A(s_t, a_t)\right]$$

这个期望时沿着轨迹 $\tau = (s_0, a_0, s_1, a_1, \cdots)$ 求得的，而 τ 是由 $\overline{\pi}$ 生成的。

证明. 首先利用一布转移概率：

$$Q_\pi(s,a) = \mathbb{E}_{s' \sim P(\cdot \mid s,a)}[r(s) + \gamma V_\pi(s^{'})]$$

于是优势函数可以表示为

$$\begin{aligned} A_\pi(s,a) &= Q_\pi(s,a) - V_\pi(s) \\ &= \mathbb{E}_{s' \sim P(s' \mid s,a)}[r(s) + \gamma V_\pi(s^{'})] - V_\pi(s) \\ &= \mathbb{E}_{s' \sim P(s' \mid s,a)}[r(s) + \gamma V_\pi(s^{'}) - V_\pi(s)] \end{aligned}$$

于是

$$\begin{aligned}\sum_{t=0}^{\infty}\gamma^t A_\pi(s_t,a_t) &= \sum_{t=0}^{\infty}\gamma^t \mathbb{E}_{s_{t+1}\sim P(s_{t+1}|s_t,a_t)}[r(s_t)+\gamma V_\pi(s_{t+1})-V_\pi(s_t)] \\ &= \sum_{t=0}^{\infty}\gamma^t r(s_t)+\sum_{t=0}^{\infty}\gamma^{t+1}\mathbb{E}_{s_{t+1}\sim P(s_{t+1}|s_t,a_t)}[V_\pi(s_{t+1})]-\sum_{t=0}^{\infty}\gamma^t V_\pi(s_t)\end{aligned}$$

根据重期望公式：

$$\mathbb{E}_{\tau\sim\overline{\pi}}\left[\sum_{t=0}^{\infty}\gamma^{t+1}\mathbb{E}_{s_{t+1}\sim P(s_{t+1}|s_t,a_t)}[V_\pi(s_{t+1})]\right]=\mathbb{E}_{\tau\sim\overline{\pi}}\left[\sum_{t=1}^{\infty}\gamma^t V_\pi(s_t)\right]$$

于是

$$\begin{aligned}\mathbb{E}_{\tau\sim\overline{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t A_\pi(s_t,a_t)\right] &= \mathbb{E}_{\tau\sim\overline{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t r(s_t)+\sum_{t=1}^{\infty}\gamma^t V_\pi(s_t)-\sum_{t=0}^{\infty}\gamma^t V_\pi(s_t)\right] \\ &= \mathbb{E}_{\tau\sim\overline{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t r(s_t)-V_\pi(s_0)\right]\end{aligned}$$

其中一项就是 $\eta(\overline{\pi})$ 的定义，第二项中的 $V_\pi(s_0)$ 是关于 s_0 的随机变量，所以

$$\begin{aligned}\mathbb{E}_{\tau\sim\overline{\pi}}[V_\pi(s_0)] &= \mathbb{E}_{s_0\sim\rho_0}[V_\pi(s_0)] \\ &= \mathbb{E}_{s_0\sim\rho_0}\left[\mathbb{E}_{a_0\sim\pi(a_0|s_0),s_1\sim P(s_1|s_0,a_0)}\left[\sum_{t=0}^{\infty}\gamma^t r(s_t)\mid s_0\right]\right] \\ &= \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t r(s_t)\right]=\eta(\pi)\end{aligned}$$

故

$$\eta(\overline{\pi})=\eta(\pi)+\mathbb{E}_{\tau\sim\overline{\pi}}\left[\sum_{t=0}^{\infty}\gamma^t A(s_t,a_t)\right]$$

□

如果把 $\overline{\pi}$ 看作是由 π 更新得到的新策略，那么上述定义告诉了我们一件事：可以用新策略产生轨迹上求得的折扣优势函数来衡量新策略相较于旧策略是否更优 (用值函数评估)。将其展开：

$$\begin{aligned}\eta(\overline{\pi}) &= \eta(\pi)+\sum_{t=0}^{\infty}\sum_{s}P(s_t=s\mid\overline{\pi})\sum_{a}\overline{\pi}(a\mid s)\gamma^t A_\pi(s,a) \\ &= \eta(\pi)+\sum_{s}\sum_{t=0}^{\infty}\gamma^t P(s_t=s\mid\overline{\pi})\sum_{a}\overline{\pi}(a\mid s)A_\pi(s,a) \\ &= \eta(\pi)+\sum_{s}\rho_{\overline{\pi}}(s)\sum_{a}\overline{\pi}(a\mid s)A_\pi(s,a)\end{aligned}$$

其中 $\rho_{\overline{\pi}}(s) = P(s_0 = s) + \gamma P(s_1 = s) + \gamma^2 P(s_2 = s) + \cdots$。要想 $\eta(\overline{\pi}) \geq \eta(\pi)$，必须满足 $\overline{\pi}(a \mid s) A_\pi(s, a) \geq 0$。传统的确定性策略改进方法：

$$\overline{\pi}(s) = \arg\max_a Q_\pi(s, a)$$

是满足 $\overline{\pi}(a \mid s) A_\pi(s, a) \geq 0$ 这个条件的，所以每一步优化策略至少是不减的。

11.2.3 MM 算法-第一次近似

但是由于 $\rho(\overline{\pi})$ 难以计算，我们介绍一个 η 的局部近似：

$$L_\pi(\overline{\pi}) = \eta(\pi) + \sum_s \rho_\pi(s) \sum_a \overline{\pi}(a \mid s) A_\pi(s, a).$$

其中 $\rho_\pi(s)$ 是访问频率。如果 $\pi_\theta(a \mid s)$ 是一个函数化策略，并且 $\pi_\theta(a \mid s)$ 关于参数 θ 可微分，那么对于任意参数 θ_0

$$\begin{aligned} L_{\pi_{\theta_0}}(\pi_{\theta_0}) &= \eta(\pi_{\theta_0}), \\ \nabla_\theta L_{\theta_0}(\pi_\theta) \mid_{\theta=\theta_0} &= \nabla_\theta \eta(\pi_\theta) \mid_{\theta=\theta_0} \end{aligned}$$

证明. 首先

$$\begin{aligned} L_{\pi_{\theta_0}}(\pi_{\theta_0}) &= \eta(\pi_{\theta_0}) + \sum_s \rho_{\pi_{\theta_0}}(s) \sum_a \pi_{\theta_0}(a \mid s) A_{\pi_{\theta_0}}(a, s) \\ &= \eta(\pi_{\theta_0}) + \sum_s \rho_{\pi_{\theta_0}}(s) \left[\sum_a \pi_{\pi_{\theta_0}}(a \mid s) Q_{\pi_{\theta_0}}(a, s) - \sum_a \pi_{\theta_0}(a \mid s) V_{\pi_{\theta_0}}(s) \right] \\ &= \eta(\pi_{\theta_0}) + \sum_s \rho_{\pi_{\theta_0}}(s) (V_{\pi_{\theta_0}}(s) - V_{\pi_{\theta_0}}(s)) \\ &= \eta(\pi_{\theta_0}) \end{aligned}$$

第二个等式求导即可。 □

这个式子说明了对于足够小的步长 $\pi_{\theta_0} \to \overline{\pi}$，改进 $L_{\pi_{\theta_{old}}}$ 的同时也会改进 η，但是我们无法求得这样的足够小的步长如何计算。下面我们介绍一个方法可以解决这个问题。

定理 11.2. 设 $D_{KL}^{\max}(\pi, \overline{\pi}) = \max_s D_{KL}(\pi(\cdot \mid s) \mid\mid \overline{\pi}(\cdot \mid s)), \epsilon = \max_{s,a} |A_\pi(s, a)|$，则误差下界为

$$\eta(\overline{\pi}) \geq L_\pi(\overline{\pi}) - C D_{KL}^{\max}(\pi, \overline{\pi}), \quad C = \frac{4\epsilon\gamma}{(1-\gamma)^2}$$

到此为止，我们可以设计一个 MM 算法来优化 $\eta(\pi)$。设 $M_i(\pi) = L_{\pi_i}(\pi) - CD_{KL}^{\max}(\pi_i, \pi)$，则

$$\eta(\pi_{i+1}) \geq M_i(\pi_{i+1})$$

$$\eta(\pi_i) = M_i(\pi_i)$$

$$\eta(\pi_{i+1}) - \eta(\pi_i) \geq M_i(\pi_{i+1}) - M(\pi_i)$$

其中

$$\pi_{i+1} = \arg\max_{\pi} M_i(\pi)$$

可以通过迭代下界函数来使 $\eta(\overline{\pi})$ 单调上升，所以上述问题转化为求解：

$$\max_{\pi_\theta}[L_{\pi_{\theta_{old}}}(\pi_\theta) - CD_{KL}^{\max}(\pi_{\theta_{old}}, \pi_\theta)]$$

11.2.4 求解上述最优化问题–第二次近似

由于优化问题：

$$\max_{\pi_\theta}[L_{\pi_{\theta_{old}}}(\pi_\theta) - CD_{KL}^{\max}(\pi_{\theta_{old}}, \pi_\theta)]$$

的惩罚系数 C 太大了 *(通常 $\gamma = 0.95 \sim 0.995$ 之间)*，这会导致步长很小从而使得训练速度很慢。为了解决这个问题，我们将惩罚项转换为约束：

$$\max_{\pi_\theta} L_{\pi_{\theta_{old}}}(\pi_\theta), \quad s.t. D_{KL}^{\max}(\pi_{\theta_{old}}, \pi_\theta) \leq \delta.$$

这样的话，在利用拉格朗日乘子法解决上述带约束的优化问题时，其参数 λ 会根据约束 δ 自适应的改变，从而提升训练速度。上述的约束就是策略参数的置信区间，也就是 *TRPO* 名称的由来。他既解决了惩罚系数太大导致训练速度慢的问题，又将更新步长限制在 δ 之内，从而将步长限制在一个合理的区间。

但是上述的 max 约束需要我们遍历所有状态的 KL 散度，这会使得在实际计算量变得很大。使用期望表示的平均 KL 散度可以让我们使用批量样本去近似 *(蒙特卡罗模拟)*，定义：

$$\overline{D}_{KL}^{\rho}(\theta_1, \theta_2) := \mathbb{E}_{s\sim\rho}[D_{KL}(\pi_{\theta_1}(\cdot \mid s), \pi_{\theta_2}(\cdot \mid s))]$$

故上述优化问题等效于：

$$\max_{\pi_\theta} L_{\pi_{\theta_{old}}}(\theta), \quad s.t. \overline{D}_{KL}^{\rho}(\pi_{\theta_{old}}, \pi_\theta) \leq \delta.$$

11.2.5 蒙特卡罗模拟去近似优化问题中带有期望的式子——第三处近似

将上述优化问题展开：

$$\max_{\pi_\theta} \sum_s \rho_{\pi_{\theta_{old}}}(s) \sum_a \pi_\theta(a \mid s) A_{\pi_{\theta_{old}}}(s,a),$$

$$s.t.\, \overline{D}_{KL}^{\rho}(\pi_{\theta_{old}}, \pi_\theta) \le \delta.$$

$L_{\pi_{\theta_{old}}}$ 中原本的 $\eta(\pi_{\theta_{old}})$ 与参数 θ 无关，可以省略。并且

$$\begin{aligned}
&\sum_s \rho_{\pi_{\theta_{old}}}(s) \sum_a \pi_\theta(a \mid s) A_{\pi_{\theta_{old}}}(s,a) \\
=&\sum_s \rho_{\pi_{\theta_{old}}}(s) \sum_a \pi_\theta(a \mid s)(Q_{\pi_{\theta_{old}}}(s,a) - V_{\pi_{\theta_{old}}}(s)) \\
=&\sum_s \rho_{\pi_{\theta_{old}}}(s) \sum_a \pi_\theta(a \mid s) Q_{\pi_{\theta_{old}}}(s,a) - \sum_s \rho_{\pi_{\theta_{old}}}(s) \sum_a \pi_\theta(a \mid s) V_{\pi_{\theta_{old}}}(s) \\
=&\sum_s \rho_{\pi_{\theta_{old}}}(s) \sum_a \pi_\theta(a \mid s) Q_{\pi_{\theta_{old}}}(s,a) - \underbrace{\sum_s \rho_{\pi_{\theta_{old}}}(s) V_{\pi_{\theta_{old}}}(s)}_{\text{常数}}
\end{aligned}$$

所以原问题可以进一步简化为：

$$\max_{\pi_\theta} \sum_s \rho_{\pi_{\theta_{old}}}(s) \sum_a \pi_\theta(a \mid s) Q_{\pi_{\theta_{old}}}(s,a),$$

$$s.t.\, \overline{D}_{KL}^{\rho}(\pi_{\theta_{old}}, \pi_\theta) \le \delta.$$

为了使用蒙特卡罗模拟，我们需要将上述目标函数转化为期望的形式。首先用 $\frac{1}{1-\gamma}\mathbb{E}_{s\sim\rho_{\pi_{\theta_{old}}}}[\cdots]$ 去替代 $\sum_s \rho_{\pi_{\theta_{old}}}(s)[\cdots]$。这一步乘以系数 $\frac{1}{1-\gamma}$ 是为了归一化处理。我们无法直接从后面的 $\pi_\theta(a \mid s)$ 中直接采样，这里我们采取重要性采样，用另一个简单的分布 $q(a \mid s)$ 去近似期望。综上所述，上述式子的期望形式为：

$$\max_{\pi_\theta} \mathbb{E}_{s\sim\rho_{\pi_{\theta_{old}}}, a\sim q}\left[\frac{\pi_\theta(a \mid s)}{q(a \mid s)} Q_{\pi_{\theta_{old}}}(s,a)\right]$$

$$s.t.\, \mathbb{E}_{s\sim\rho_{\theta_{old}}}[D_{KL}(\pi_{\theta_{old}}(\cdot \mid s) \,||\, \pi_\theta(\cdot \mid s))] \le \delta.$$

接下来比较重要的环节就是选择合理的采样方式。

11.2.6 求解采样之后的优化问题

这一部分和之前的自然梯度下降一样。

11.3 近端策略梯度下降

上一节讲的 *TRPO* 由于算法设计过于复杂，普适性不强，接下来介绍的近端策略梯度下降才是强化学习当前的热门算法。首先选取 $\pi_{\theta_{old}}$ 作为采样策略，那么上一节中的优化问题变为：

$$\max_{\pi_\theta} \mathbb{E}_{s\sim\rho_{\pi_{\theta_{old}}},a\sim\pi_{\theta_{old}}}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\theta_{old}}(a\mid s)}A_{\pi_{\theta_{old}}}(s,a)\right]$$
$$s.t.\ \mathbb{E}_{s\sim\rho_{\pi_{\theta_{old}}}}[D_{KL}(\pi_{\theta_{old}}(\cdot\mid s)\mid\mid\pi_\theta(\cdot\mid s))]\le\delta.$$

记*:*

$$r_t(\theta)=\frac{\pi_\theta(a\mid s)}{\pi_{\theta_{old}}(a\mid s)},\quad r_t(\theta_{old})=1.$$

这个比率表示新旧策略采样的可能性，如果 $r_t(\theta)$ 偏离 1 的程度越大，说明策略更新程度越高。*PPO* 不使用 *KL* 散度来限制策略更新的程度，而是通过限制 $r_t(\theta)$ 的大小来限制更新程度。这样可以避免使用二阶优化 *(TRPO* 以及 *NPO* 中的二阶优化均来自与 *KL* 散度对策略更新程度的约束*)*。于是 *PPO* 算法实际上是解决如下优化问题：

$$\max_{\pi_\theta}\mathbb{E}_{s\sim\rho_{\pi_{\theta_{old}}},a\sim\pi_{\theta_{old}}}[\min\{r_t(\theta)A_{\pi_{\theta_{old}}}(s,a),\mathrm{clip}(r_t(\theta),1{+}\epsilon,1{-}\epsilon)A_{\pi_{\theta_{old}}}(s,a)\}]$$

这时我们便可以使用一阶优化方法进行计算，例如 *Adam* 和 *SGD*。

11.3.1 Clip 就是隐式 KL 散度

下面的证明说明了 *Clip-PPO* 拥有隐式 *KL* 散度*,* 这也就是说 *PPO* 和 *TRPO* 的实际效果是一样的，而 *PPO* 能使用一阶优化，*TRPO* 只能使用二阶优化。

根据 *KL* 散度定义：

$$\begin{aligned}D_{KL}(\pi_{\theta_{old}}\mid\mid\pi_\theta)&=\int\pi_{\theta_{old}}(a\mid s_t)\log\frac{\pi_{\theta_{old}}(a\mid s_t)}{\pi_\theta(a\mid s_t)}\\&=\mathbb{E}_{a\sim\pi_{\theta_{old}}}[-\log r_t(\theta)]\end{aligned}$$

由于 *clip* 的限制，$1-\epsilon\le r_t(\theta)\le 1+\epsilon$，所以 *KL* 散度的上界为：

$$D_{KL}(\pi_{\theta_{old}}\mid\mid\pi_\theta)\le-\log(1-\epsilon)$$

在实际应用中超参数 ϵ 大多情况下比较小，所以 $-\log(1-\epsilon)$ 可以近似为：

$$-\log(1-\epsilon)\approx\epsilon+\frac{\epsilon}{2}$$

故 *KL* 散度有个近似上界*:*

$$D_{KL}(\pi_{\theta_{old}}\mid\mid\pi_\theta)\lesssim\epsilon$$

这就说明在使用 *Clip-PPO* 时，其实上也将 *KL* 散度限制在了 ϵ 之内。